

Data Preprocessing For Datasets

Group 5

2026-09-18

Table of contents

Loading the packages we need	2
File paths for the dataset folders	3
1. Data Import, Structure and Selection	3
1.1 Importing the datasets with the correct column types	3
1.2 Summary of the datasets before cleaning	5
1.3 Checking the grain and keys of each dataset	8
1.4 Why the other seven datasets are not used	11
2. Data Cleaning	17
2.1 Recovering the missing household IDs in D01	18
2.2 Finding and removing duplicate records	21
2.3 Text placeholders in code columns	24
2.4 Filling gaps using the other datasets	27
Age in D01	27
Education level in D02	30
3. Column Selection	31
Columns that add no information or repeat another column	32
Columns that give away the employment outcome	35
Columns that cannot be used or are not needed	36
Education columns for Link A (digital access → education)	38
Keeping the selected columns	40
4. Special Values	46
4.1 Classifying the special codes	47
4.2 Placeholder values in salary	51
5. Record Selection (ages 15–64)	53

6. Data Transformation: Recoding Codes into Labelled Factors	56
7. Outlier Review	65
7.1 Age	66
7.2 Household smartphones	68
7.3 Salary	73
8. Validating and Saving the Cleaned Datasets	77
8.1 Checking the cleaned datasets	77
8.2 Saving the cleaned datasets	85
9. Data Integration	89
9.1 Checking keys and cardinality before joining	89
9.2 Joining the datasets	92
9.3 Checking the integrated dataset	95
10. Feature Engineering, Transformation and Encoding	97
10.1 Removing the flag columns	98
10.2 Recoding yes/no columns to 0/1	98
10.3 Engineered features	100
10.3.1 Education: <code>education_band</code> and <code>matric_plus</code>	101
10.3.2 Digital access: <code>any_home_internet</code> and <code>digital_access_score</code>	103
10.3.3 Education × digital access: <code>edu_digital_group</code>	105
10.3.4 Location and age group: <code>settlement_type</code> and <code>youth</code>	107
10.3.5 Household features	109
10.4 Transformations	113
T1 Target population: the labour force	113
T2 Centred age and age squared	113
T6 Log salary (income question only)	115
T3, T4, T5 and leakage: building the modelling dataset	116
T7 Why no scaling is applied	121
10.5 Encoding for modelling	122
10.6 Saving the analytical datasets	125

Loading the packages we need

```
library(readr)
library(tidyverse)
library(janitor)
library(dplyr)
```

File paths for the dataset folders

```
directory <- "../Datasets/"
cleaned_directory <- "../Datasets/Cleaned/" #Cleaned Datasets get stored here
dir.create(cleaned_directory, showWarnings = FALSE)
```

1. Data Import, Structure and Selection

1.1 Importing the datasets with the correct column types

Before anything can be cleaned, the four datasets have to be read in correctly. If R is left to guess the column types, a few columns lose information on import:

- **HouseholdID** is an 18-digit number. R stores numbers as doubles, which are only exact up to about 15–16 digits, so reading the IDs as numbers rounds them and different households can end up with the same ID (Milestone 1, section 8.3).
- **PersonNumber** is stored as “01”, “02”, and so on. It needs the same type in every file, otherwise the joins will not match.
- **EducationAttendanceCode** and **MinutesToWork** have text placeholders mixed in with their numeric codes, and these would be lost if the columns were read as numbers.

To avoid this, each file is read with its types set explicitly. The IDs are read as text (`col_character()`) in all four files, and the two mixed columns are also read as text so that no value is silently dropped. `clean_names()` then converts all column names to snake_case. The datasets keep the names D01, D02, D03 and D08, as in the ERD and the EDA, and are cleaned step by step under those same names. Section 1.2 then records what each dataset looks like before anything changes, so every later step can be compared with the starting point.

```
D01 <- read_csv(
  file.path(directory, "D01_Population_Profile.csv"),
  show_col_types = FALSE,
  col_types = cols(HouseholdID = col_character(), PersonNumber = col_character())
) |> clean_names()

D02 <- read_csv(
  file.path(directory, "D02_Learning_Profile.csv"),
  show_col_types = FALSE,
  col_types = cols(HouseholdID = col_character(), PersonNumber = col_character(),
    EducationAttendanceCode = col_character(), MinutesToWork = col_character())
) |> clean_names()
```

Educate

```

D03 <- read_csv(
  file.path(directory, "D03_Economic_Participation.csv"),
  show_col_types = FALSE,
  col_types = cols(HouseholdID = col_character(),
    PersonNumber = col_character(),
    MinutesToWork = col_character())
) |> clean_names()

D08 <- read_csv(
  file.path(directory, "D08_Connectivity_Access.csv"),
  show_col_types = FALSE,
  col_types = cols(HouseholdID = col_character())
) |> clean_names()

```

The table below confirms that the import worked: there are no parsing problems, and the IDs are stored as text with all 18 digits intact. The last column shows what would have happened if the IDs had been read as numbers instead.

```

#We create a display table (tibble) that gives important information about each dataset after
tibble(

  dataset = c("D01", "D02", "D03", "D08"),

  parsing_problems = c(
    nrow(problems(D01)),
    nrow(problems(D02)),
    nrow(problems(D03)),
    nrow(problems(D08))
  ),

  household_id_type = c(
    class(D01$household_id),
    class(D02$household_id),
    class(D03$household_id),
    class(D08$household_id)
  ),

  shortest_id = c(
    min(nchar(D01$household_id), na.rm = TRUE),
    min(nchar(D02$household_id)),
    min(nchar(D03$household_id)),

```

```

    min(nchar(D08$household_id))
  ),

  longest_id = c(
    max(nchar(D01$household_id), na.rm = TRUE),
    max(nchar(D02$household_id)),
    max(nchar(D03$household_id)),
    max(nchar(D08$household_id))
  ),

  distinct_households = c(
    n_distinct(D01$household_id, na.rm = TRUE),
    n_distinct(D02$household_id),
    n_distinct(D03$household_id),
    n_distinct(D08$household_id)),

  distinct_if_stored_as_number = c(
    n_distinct(as.numeric(D01$household_id), na.rm = TRUE),
    n_distinct(as.numeric(D02$household_id)),
    n_distinct(as.numeric(D03$household_id)),
    n_distinct(as.numeric(D08$household_id))
  )
)

```

dataset	pars- ing_prob- lems	house- hold_id_type	short- est_id	longest_id	dis- tinct_house- holds	dis- tinct_if_stored_as_num- ber
D01	0	character	18	18	20940	20684
D02	0	character	18	18	20940	20684
D03	0	character	18	18	20940	20684
D08	0	character	18	18	20940	20684

Stored as numbers, D08's 20,940 distinct households would collapse to 20,684, merging different households into one. This is exactly the count behind the EDA's 98.77% household match, so that figure was caused by the rounding and not by missing households. Read as text, every ID keeps all 18 digits.

1.2 Summary of the datasets before cleaning

To show what each cleaning step changes, we first need a record of what the data looked like at the start. `uncleaned_summary` stores the size, missing values, number of distinct keys and

exact duplicate rows of each dataset before any cleaning is done, and later sections compare the cleaned data against this table.

```
uncleaned_summary <- tibble(  
  
  dataset = c("D01", "D02", "D03", "D08"),  
  
  rows = c(  
    nrow(D01),  
    nrow(D02),  
    nrow(D03),  
    nrow(D08)  
  ),  
  
  cols = c(  
    ncol(D01),  
    ncol(D02),  
    ncol(D03),  
    ncol(D08)  
  ),  
  
  missing_values = c(  
    sum(is.na(D01)),  
    sum(is.na(D02)),  
    sum(is.na(D03)),  
    sum(is.na(D08))  
  ),  
  
  missing_household_id = c(  
    sum(is.na(D01$household_id)),  
    sum(is.na(D02$household_id)),  
    sum(is.na(D03$household_id)),  
    sum(is.na(D08$household_id))  
  ),  
  
  distinct_keys = c(  
    nrow(distinct(D01, household_id, person_number)),  
    nrow(distinct(D02, household_id, person_number)),  
    nrow(distinct(D03, household_id, person_number)),  
    n_distinct(D08$household_id)  
  ),  
  
  exact_duplicate_rows = c(  

```

```

    nrow(D01) - nrow(distinct(D01)),
    nrow(D02) - nrow(distinct(D02)),
    nrow(D03) - nrow(distinct(D03)),
    nrow(D08) - nrow(distinct(D08))
  )
)
uncleaned_summary

```

dataset	rows	cols	missing_val- ues	missing_house- hold_id	dis- tinct_keys	exact_dupli- cate_rows
D01	70440	17	115	25	70422	0
D02	70462	23	120	0	70440	22
D03	70460	25	100	0	70440	20
D08	20940	18	60	0	20940	0

The next table lists every column that has missing values in the uncleaned data, along with the percentage of rows affected:

```

bind_rows(
  D01 = enframe(colSums(is.na(D01)), name = "column", value = "missing"),
  D02 = enframe(colSums(is.na(D02)), name = "column", value = "missing"),
  D03 = enframe(colSums(is.na(D03)), name = "column", value = "missing"),
  D08 = enframe(colSums(is.na(D08)), name = "column", value = "missing"),
  .id = "dataset"
) |>

filter(missing > 0) |>
left_join(select(uncleaned_summary, dataset, rows), by = "dataset") |>
mutate(percent_of_rows = round(missing / rows * 100, 2))

```

dataset	column	missing	rows	percent_of_rows
D01	household_id	25	70440	0.04
D01	age	90	70440	0.13
D02	education_level_code	120	70462	0.17
D03	employment_status1code	100	70460	0.14
D08	mobile_internet_code	60	20940	0.29

1.3 Checking the grain and keys of each dataset

Before any datasets are joined, we need to confirm what one row represents in each dataset (its grain) and which columns uniquely identify a row (its key), and show that the datasets really describe the same households. The first table compares the number of rows with the number of distinct keys:

```
D01_repeated_keys <- D01 |>
  filter(!is.na(household_id)) |>
  count(household_id, person_number) |>
  filter(n > 1) |>
  nrow()

D02_repeated_keys <- D02 |>
  count(household_id, person_number) |>
  filter(n > 1) |>
  nrow()

D03_repeated_keys <- D03 |>
  count(household_id, person_number) |>
  filter(n > 1) |>
  nrow()

D08_repeated_keys <- D08 |>
  count(household_id) |>
  filter(n > 1) |>
  nrow()

tibble(

  dataset = c("D01", "D02", "D03", "D08"),

  grain = c("person", "person", "person", "household"),

  key = c(
    "household_id + person_number", "household_id + person_number",
    "household_id + person_number", "household_id"
  ),

  rows = c(
    nrow(D01),
    nrow(D02),
```

```

nrow(D03),
nrow(D08)
),

distinct_keys = c(
  nrow(distinct(filter(D01, !is.na(household_id)), household_id, person_number)),
  nrow(distinct(D02, household_id, person_number)),
  nrow(distinct(D03, household_id, person_number)),
  n_distinct(D08$household_id)
),

keys_appearing_more_than_once = c(
  D01_repeated_keys,
  D02_repeated_keys,
  D03_repeated_keys,
  D08_repeated_keys
),

rows_missing_household_id = c(
  sum(is.na(D01$household_id)),
  sum(is.na(D02$household_id)),
  sum(is.na(D03$household_id)),
  sum(is.na(D08$household_id))
)
)

```

dataset	grain	key	rows	dis- tinct_keys	keys__appear- ing_more_than_once	rows_miss- ing_house- hold_id
D01	per- son	household_id + person_number	70440	70415	0	25
D02	per- son	household_id + person_number	70462	70440	22	0
D03	per- son	household_id + person_number	70460	70440	20	0
D08	house- hold	household_id	20940	20940	0	0

The next table counts how many of D01's households also appear in each of the other selected datasets:

```

D01_households <- unique(na.omit(D01$household_id)) #Distinct household IDs in D01

tibble(

  compared_with = c("D02", "D03", "D08"),

  households_in_dataset = c(
    n_distinct(D02$household_id),
    n_distinct(D03$household_id),
    n_distinct(D08$household_id)
  ),

  shared_with_D01 = c(
    length(intersect(D01_households, D02$household_id)),
    length(intersect(D01_households, D03$household_id)),
    length(intersect(D01_households, D08$household_id))
  ),

  percent_of_D01 = round(shared_with_D01 / length(D01_households) * 100, 2)

)

```

compared_with	households_in_dataset	shared_with_D01	percent_of_D01
D02	20940	20940	100
D03	20940	20940	100
D08	20940	20940	100

Finally, the number of people per household in D01:

```

D01 |>
  filter(!is.na(household_id)) |>

  count(household_id, name = "persons") |>

  summarise(households = n(), mean_persons = round(mean(persons), 2),
            median_persons = median(persons), max_persons = max(persons))

```

households	mean_persons	median_persons	max_persons
20940	3.36	3	25

D01, D02 and D03 have one row per person, and D08 has exactly one row per household (20,940 rows and 20,940 distinct IDs). All 20,940 D01 households appear in D02, D03 and D08, a 100% match. This corrects the 98.77% reported in the EDA: its count of 20,684 was the number of distinct IDs left over after the rounding described in 1.1.

Household IDs repeat in the person files because one household contains several people. These repeats are expected in a one-to-many relationship and are not duplicates. The keys that do appear more than once in D02 and D03 are dealt with in 2.2, and the missing IDs in D01 in 2.1.

1.4 Why the other seven datasets are not used

Eleven datasets are available, but a dataset is only useful here if it passes two tests. It must measure part of the research pathway (digital access → education → employment), and it must link to the same people at the chosen grain of one row per person, identified by `household_id` + `person_number`. A dataset that fails the first test only adds variables with no role in the analysis, and one that fails the second attaches records to the wrong people.

The seven datasets that were not selected in Task 1 are read in here only to show why they are left out:

```
D04 <- read_csv(
  file.path(directory, "D04_Dwelling_Conditions.csv"),
  show_col_types = FALSE,
  col_types = cols(HouseholdID = col_character())
) |> clean_names()

D05 <- read_csv(
  file.path(directory, "D05_Basic_Services.csv"),
  show_col_types = FALSE,
  col_types = cols(HouseholdID = col_character())
) |> clean_names()

D06 <- read_csv(
  file.path(directory, "D06_Household_Energy.csv"),
  show_col_types = FALSE,
  col_types = cols(HouseholdID = col_character())
) |> clean_names()
```

```

D07 <- read_csv(
  file.path(directory, "D07_Household_Wellbeing.csv"),
  show_col_types = FALSE,
  col_types = cols(HouseholdID = col_character())
) |> clean_names()

D09 <- read_csv(
  file.path(directory, "D09_Household_Resources.csv"),
  show_col_types = FALSE,
  col_types = cols(HouseholdID = col_character())
) |> clean_names()

D10 <- read_csv(
  file.path(directory, "D10_Mobility_Access.csv"),
  show_col_types = FALSE,
  col_types = cols(HouseholdID = col_character())
) |> clean_names()

#D11 is large, so only its keys and quarter are read in full; its column names are read from
D11 <- read_csv(
  file.path(directory, "D11_Labour_Market.csv"),
  show_col_types = FALSE,
  col_select = c(HouseholdID, PersonNumber, Quarter),
  col_types = cols(.default = col_character())
) |> clean_names()

D11_columns <- read_csv(
  file.path(directory, "D11_Labour_Market.csv"),
  show_col_types = FALSE, n_max = 0
) |> clean_names() |> names()

tibble(

  dataset = c("D04", "D05", "D06", "D07", "D09", "D10", "D11"),

  measures = c(
    "Dwelling type, materials and condition",
    "Water and sanitation", "Electricity and energy use",
    "Food security and household income",
    "Income sources and household assets",
    "Public transport and access to health facilities",
    "Employment, education and occupation"
  )
)

```

```

),

grain = c("household", "household", "household", "household", "household", "household", "p

rows = c(
  nrow(D04),
  nrow(D05),
  nrow(D06),
  nrow(D07),
  nrow(D09),
  nrow(D10),
  nrow(D11)
),

distinct_households =
  c(n_distinct(D04$household_id),
    n_distinct(D05$household_id),
    n_distinct(D06$household_id),
    n_distinct(D07$household_id),
    n_distinct(D09$household_id),
    n_distinct(D10$household_id),
    n_distinct(D11$household_id)
  ),

households_shared_with_D01 = c(
  length(intersect(D01_households, D04$household_id)),
  length(intersect(D01_households, D05$household_id)),
  length(intersect(D01_households, D06$household_id)),
  length(intersect(D01_households, D07$household_id)),
  length(intersect(D01_households, D09$household_id)),
  length(intersect(D01_households, D10$household_id)),
  length(intersect(D01_households, D11$household_id))
),

reason_not_selected = c(
  "Not a link in the pathway",
  "Not a link in the pathway",
  "Not a link in the pathway",
  "Not in the pathway; household income partly reveals the employment outcome",
  "Salary income reveals employment outcome; computer_asset_code duplicates D08",
  "Not a link in the pathway",
  "Different sample: cannot be linked to any person in D01"
)

```

```
)
)
```

dataset	measures	grain	rows	dis- tinct_house- holds	house- holds_shared	reason D01 selected
D04	Dwelling type, materials and condition	household	20956	20940	20940	Not a link in the pathway
D05	Water and sanitation	household	20940	20940	20925	Not a link in the pathway
D06	Electricity and energy use	household	20940	20940	20940	Not a link in the pathway
D07	Food security and household income	household	20955	20940	20940	Not in the pathway; household income partly reveals the employment outcome
D09	Income sources and household assets	household	20955	20940	20940	Salary income reveals employment outcome; computer_asset_code duplicates D08
D10	Public transport and access to health facilities	household	20940	20940	20940	Not a link in the pathway
D11	Employment, education and occupation	person per quarter	170432	36053	0	Different sample: cannot be linked to any person in D01

D09's only digital-access column turns out to be a copy of the column already selected from D08:

```
D09 |>
  distinct(household_id, computer_asset_code_d09 = computer_asset_code) |>
  inner_join(select(D08, household_id, computer_asset_code), by = "household_id") |>
  summarise(households_compared = n(),
            values_that_differ_from_D08 = sum(
              computer_asset_code_d09 != computer_asset_code
            )
  )
```

households_compared	values_that_differ_from_D08
20940	0

D11 is the only unselected dataset that measures employment, so it gets a closer look to see whether it could be joined in any way:

```
tibble(
  check = c("rows in D11",
            "distinct people in D11",
            "people recorded in more than one quarter",
            "households shared with D01",
            "PSUs (sampling areas) shared with D01",
            "digital-access columns in D11"),

  result = c(nrow(D11),
             nrow(distinct(D11, household_id, person_number)),
             sum(count(D11, household_id, person_number)$n > 1),
             length(intersect(D11$household_id, D01_households)),
             length(intersect(str_sub(D11$household_id, 1, 11),
                                     str_sub(D01_households, 1, 11))),
             length(str_subset(D11_columns, "internet|phone|computer"))
            )
)
```

check	result
rows in D11	170432
distinct people in D11	82897
people recorded in more than one quarter	51862
households shared with D01	0
PSUs (sampling areas) shared with D01	0
digital-access columns in D11	0

```
#Levels of geography at which D11 could be linked to the four selected datasets
selected_columns <- c(
  names(D01),
  names(D02),
  names(D03),
  names(D08)
)
```

```

tibble(

  geography_level = c(
    "province",
    "geography type",
    "metro area"
  ),

  column_in_D11 = c(
    any(str_detect(D11_columns, "province")),
    any(str_detect(D11_columns, "geo_type")),
    any(str_detect(D11_columns, "metro"))
  ),

  column_in_selected_datasets = c(
    any(str_detect(selected_columns, "province")),
    any(str_detect(selected_columns, "geo_type")),
    any(str_detect(selected_columns, "metro"))
  )
)

```

geography_level	column_in_D11	column_in_selected_datasets
province	TRUE	TRUE
geography type	TRUE	TRUE
metro area	FALSE	TRUE

The only geography D11 has in common with the selected datasets is province and geography type. It has no metro column, and none of its PSUs match D01's (first table above), so province and geography type are the most detailed levels at which D11 could ever be linked to our people.

Based on these checks, only D01, D02, D03 and D08 are kept, and the other seven are removed from memory:

- **D04, D05, D06, D07, D09 and D10** are household-level datasets that share almost every household with D01, so technically they could be joined many-to-one. They are left out because of the topic, not because they cannot be linked. Dwelling conditions, water, energy and public transport are not part of the path from digital access through education to employment. Household income (D07, D09) and salary income sources (D09) partly reveal whether someone in the household is employed, so they would leak the outcome, and D09's only digital-access column is a copy of D08's. D04, D07 and D09

also contain repeated household rows, which would duplicate people if they were joined as they are.

- **D11** would have been useful, since it records employment status and education for working-age people, but it cannot be joined at the person grain. It shares no households and not a single PSU with D01, which means it is a separate sample of different people rather than extra information about ours. Its rows are also person-quarters rather than people: most people appear in more than one quarter, with separate quarterly and annual weights. The only way to join it would be to match people on shared characteristics such as province, age and sex, which would attach one person's labour-market record to someone else entirely.
- **Aggregating D11 does not help either.** D11 only shares province and geography type with D01. A D11 rate calculated per province and attached to each person would be fully determined by `province_code`, which D01 already has, so it adds nothing at the person level. D11 also has no digital-access columns, so a separate analysis could only repeat the education → employment link and could not look at digital access or its moderating role.

```
rm(  
  D04,  
  D05,  
  D06,  
  D07,  
  D09,  
  D10,  
  D11,  
  D11_columns,  
  selected_columns  
) #Only needed for the checks above
```

The tables above back up this decision. All six household datasets share almost all of D01's households, so leaving them out is a choice about the topic. D09's computer column never differs from D08's, and D11 shares 0 households and 0 PSUs with D01. From here on, the pipeline only cleans and integrates D01, D02, D03 and D08. This does mean the employment outcome comes from a single source (D03), which is noted as a limitation.

2. Data Cleaning

This section fixes the problems found in the uncleaned data: missing IDs, duplicate records, text placeholders in code columns, and missing or invalid values that can be filled in from another dataset.

No value is changed without leaving a note. Whenever a step changes a value, it adds a column ending in `_flag` that sits next to the data, such as `age_flag` for `age`. On every row that was changed, the flag column holds a short note saying what happened (for example “missing - filled from D03”), and on every other row it is empty. This shows exactly which values were changed and why, and lets the checks count them. The same idea is used later to mark impossible smartphone counts and implausibly low salaries. The flag columns only keep track of the cleaning and are not part of the data itself, so they are left out of the saved files and removed before modelling.

2.1 Recovering the missing household IDs in D01

25 rows in D01 have no `household_id`, so these people cannot be linked to their education, employment or household connectivity. Exactly the same number of people appear in D03 (and D02) but not in D01, which suggests they are the same people with their ID blanked out.

To recover the IDs, each D01 row without an ID is matched to one of the people missing from D01 using `person_number`, `sex_code`, `province_code` and `person_weight`. The survey weight is a strong column to match on: it is identical for the same person across D01–D03 and has thousands of distinct values. Each match is then double-checked with two columns that were **not** used for matching: the PSU (sampling cluster) must equal the first 11 digits of the recovered household ID, and the age must agree with D03.

```
D01_key_duplicates_before <- get_dupes(D01, household_id, person_number)

missing_from_D01 <- D03 |>
  distinct() |>
  anti_join(
    filter(D01, !is.na(household_id)),
    by = c("household_id", "person_number")
  ) |>
  select(
    household_id,
    person_number,
    sex_code,
    province_code,
    person_weight,
    age_d03 = age
  )

rows_missing_id <- D01 |>
  mutate(row_id = row_number()) |>
  filter(is.na(household_id)) |>
```

```

select(-household_id)

recovered <- rows_missing_id |>
  inner_join(
    missing_from_D01,
    by = c("person_number", "sex_code", "province_code", "person_weight")
  )

```

The table below shows that the matching is reliable. Every D01 row without an ID matches exactly one person, every person is used only once, and both independent checks agree for every match.

```

tibble(

  D01_rows_missing_id = nrow(rows_missing_id),

  people_missing_from_D01 = nrow(missing_from_D01),

  matches_found = nrow(recovered),

  each_d01_row_matched_once = n_distinct(recovered$row_id) == nrow(rows_missing_id),

  each_person_used_once = nrow(
    distinct(recovered, household_id, person_number)
  ) == nrow(recovered),

  psu_agrees_with_id =
    sum(sprintf("%.0f", recovered$psu) == str_sub(recovered$household_id, 1, 11)),

  age_agrees_with_D03 = sum(recovered$age == recovered$age_d03, na.rm = TRUE)
)

```

D01_rows_missing_id	people_missing_from_D01	matches_found	each_d01_row_matched_once	each_person_used_once	psu_agrees_with_id	age_agrees_with_D03
25	25	25	TRUE	TRUE	25	25

With the matches confirmed, the recovered IDs are written back into D01, and `household_id_flag` notes which rows were recovered:

```

D01 <- D01 |>
  mutate(row_id = row_number()) |>
  left_join(
    select(recovered, row_id, recovered_id = household_id), by = "row_id"
  ) |>
  mutate(
    household_id_flag =
      if_else(
        is.na(household_id) & !is.na(recovered_id),
        "recovered by matching to D03", NA_character_,
      ),
    household_id = coalesce(household_id, recovered_id)
  ) |>
  select(-row_id, -recovered_id)

tibble(

  missing_household_id_before = uncleaned_summary$missing_household_id[1],

  missing_household_id_after = sum(is.na(D01$household_id)),

  rows_before = uncleaned_summary$rows[1],

  rows_after = nrow(D01),

  D01_keys_identical_to_D02 = setequal(
    paste(D01$household_id, D01$person_number),
    paste(D02$household_id, D02$person_number)
  )
)

```

missing_house- hold_id_before	missing_house- hold_id_after	rows_be- fore	rows_af- ter	D01_keys_identi- cal_to_D02
25	0	70440	70440	TRUE

All 25 IDs are recovered without adding or removing any rows, and D01 now contains exactly the same people as D02. The recovered rows are the only rows with a note in `household_id_flag`.

2.2 Finding and removing duplicate records

The EDA found 23 (D01), 44 (D02) and 40 (D03) rows sharing a person key. Some of these could be real duplicates (the same record entered twice), while others could be legitimate repeats, so the two need to be told apart before anything is removed.

In D01, every one of these “duplicates” had a missing household ID. Rows with no ID and the same person number were grouped together even though they are different people, so they were never real duplicates:

```
D01_key_duplicates_before |>
  count(household_id_missing = is.na(household_id), name = "rows")
```

household_id_missing	rows
TRUE	23

Now that the IDs are recovered (2.1), each dataset is checked again for two things: exact copies, where every column is identical to another row, and keys that still repeat with different values, which would mean conflicting records for the same person:

```
#Rows that share their key with at least one other row
D01_key_duplicate_rows <- nrow(get_dupes(D01, household_id, person_number))
D02_key_duplicate_rows <- nrow(get_dupes(D02, household_id, person_number))
D03_key_duplicate_rows <- nrow(get_dupes(D03, household_id, person_number))
D08_key_duplicate_rows <- nrow(get_dupes(D08, household_id))

#Exact copies: rows where every column is identical to another row
D01_exact_duplicates <- nrow(D01) - nrow(distinct(D01))
D02_exact_duplicates <- nrow(D02) - nrow(distinct(D02))
D03_exact_duplicates <- nrow(D03) - nrow(distinct(D03))
D08_exact_duplicates <- nrow(D08) - nrow(distinct(D08))

#Keys that still repeat once exact copies are removed, meaning the same person with conflict
D01_conflicting_keys <- distinct(D01) |>
  count(household_id, person_number) |>
  filter(n > 1) |>
  nrow()

D02_conflicting_keys <- distinct(D02) |>
  count(household_id, person_number) |>
  filter(n > 1) |>
```

```

nrow()

D03_conflicting_keys <- distinct(D03) |>
  count(household_id, person_number) |>
  filter(n > 1) |>
  nrow()

D08_conflicting_keys <- distinct(D08) |>
  count(household_id) |>
  filter(n > 1) |>
  nrow()

tibble(

  dataset = c("D01", "D02", "D03", "D08"),

  rows_before = c(
    nrow(D01),
    nrow(D02),
    nrow(D03),
    nrow(D08)
  ),

  rows_in_key_duplicate_groups = c(
    D01_key_duplicate_rows,
    D02_key_duplicate_rows,
    D03_key_duplicate_rows,
    D08_key_duplicate_rows
  ),

  exact_duplicate_rows = c(
    D01_exact_duplicates,
    D02_exact_duplicates,
    D03_exact_duplicates,
    D08_exact_duplicates
  ),

  conflicting_keys_after_removing_exact_copies = c(
    D01_conflicting_keys,
    D02_conflicting_keys,
    D03_conflicting_keys,
    D08_conflicting_keys
  )
)

```

```
)
)
```

dataset	rows_before	rows_in_key_duplicate_groups	exact_duplicate_rows	conflicting_keys_after_removing_exact_copies
D01	70440	0	0	0
D02	70462	44	22	0
D03	70460	40	20	0
D08	20940	0	0	0

Every repeated key in D02 and D03 is an exact copy of another row, with no conflicting values, so these are true duplicate records and can safely be removed with `distinct()`. D01 and D08 contain no exact copies.

```
D01 <- distinct(D01)
D02 <- distinct(D02)
D03 <- distinct(D03)
D08 <- distinct(D08)

tibble(
  dataset = c("D01", "D02", "D03", "D08"),

  rows_before = uncleaned_summary$rows,

  rows_after = c(
    nrow(D01),
    nrow(D02),
    nrow(D03),
    nrow(D08)
  ),

  rows_removed = rows_before - rows_after,

  keys_unique_after = c(
    nrow(distinct(D01, household_id, person_number)) == nrow(D01),
    nrow(distinct(D02, household_id, person_number)) == nrow(D02),
    nrow(distinct(D03, household_id, person_number)) == nrow(D03),
    n_distinct(D08$household_id) == nrow(D08)
  )
)
```

dataset	rows_before	rows_after	rows_removed	keys_unique_after
D01	70440	70440	0	TRUE
D02	70462	70440	22	TRUE
D03	70460	70440	20	TRUE
D08	20940	20940	0	TRUE

D02 loses 22 rows and D03 loses 20. All three person datasets now contain exactly 70,440 people, and every key is unique in every dataset. `household_id` still repeats in the person files, which is correct, because one household contains several people.

2.3 Text placeholders in code columns

Code columns should only contain numbers, so a code column stored as text is a sign that something non-numeric has crept in. The table below lists the text columns in each dataset and why each one is text (the `_flag` columns are left out, because they hold notes and are meant to be text):

```
bind_rows(
  D01 = enframe(sapply(D01, class), name = "column", value = "type"),
  D02 = enframe(sapply(D02, class), name = "column", value = "type"),
  D03 = enframe(sapply(D03, class), name = "column", value = "type"),
  D08 = enframe(sapply(D08, class), name = "column", value = "type"),
  .id = "dataset"
) |>
filter(type == "character", !str_ends(column, "_flag")) |>
group_by(column) |>
summarise(datasets = paste(dataset, collapse = ", ")) |>
mutate(
  why_text = case_when(
    column %in% c(
      "household_id",
      "person_number") ~ "Key, read as text on purpose in 1.1",
    column == "province" ~ "Province name (rebuilt from province_code, dropped as redundant)",
    TRUE ~ "Code column containing text placeholders"
  )
)
```

column	datasets	why_text
education_attendance_code	D02	Code column containing text placeholders
household_id	D01, D02, D03, D08	Key, read as text on purpose in 1.1
minutes_to_work	D03	Code column containing text placeholders
person_number	D01, D02, D03	Key, read as text on purpose in 1.1
province	D01, D02, D03, D08	Province name (rebuilt from province_code, dropped as redundant)

Apart from the IDs and the province name, only two columns are text: `education_attendance_code` (D02, kept) and `minutes_to_work` (D03). `minutes_to_work` is dropped as leakage, so it does not need fixing. `education_attendance_code` mixes numeric codes with three text placeholders, the same problem Milestone 1 found in D11's `hours_worked_raw`:

```
D02 |> count(education_attendance_code)
```

education_attendance_code	n
1	20590
2	43540
3	20
8	6255
N/A	13
Unknown	13
not recorded	9

None of these placeholders is in the data dictionary. “not recorded” and “Unknown” are clearly non-response. It is tempting to read “N/A” as code 8 (“not applicable”), but comparing the ages shows that this does not hold. Code 8 is only used for children aged 0–4, while the placeholders appear at ages up to 72:

```
attendance_text <- c("N/A", "not recorded", "Unknown")
D02 |>
  filter(education_attendance_code %in% c("8", attendance_text)) |>
  group_by(value = education_attendance_code) |>
  summarise(
    rows = n(),
    min_age = min(age),
    median_age = median(age),
```

```
max_age = max(age)
)
```

value	rows	min_age	median_age	max_age
8	6255	0	2	4
N/A	13	8	41	72
Unknown	13	4	28	57
not recorded	9	0	19	56

Because the placeholders cannot be given a meaning from the dictionary, they are set to missing, `education_attendance_flag` notes which placeholder each row held, and the column is converted to integer.

```
D02 <- D02 |>
mutate(
  education_attendance_flag = if_else(
    education_attendance_code %in% attendance_text,
    paste0("text placeholder '", education_attendance_code, "' set to missing"),
    NA_character_),
  education_attendance_code = as.integer(if_else(
    education_attendance_code %in% attendance_text,
    NA_character_,
    education_attendance_code)
  )
)
```

The counts below show the column after the fix, along with its new type:

```
D02 |> count(education_attendance_code, education_attendance_flag)
```

education_attendance_code	education_attendance_flag	n
1	NA	20590
2	NA	43540
3	NA	20
8	NA	6255
NA	text placeholder 'N/A' set to missing	13
NA	text placeholder 'Unknown' set to missing	13

education_attendance_code	education_attendance_flag	n
NA	text placeholder 'not recorded' set to missing	9

```
class(D02$education_attendance_code)
```

```
[1] "integer"
```

`education_attendance_code` is now an integer column that only holds dictionary codes (1, 2, 3, 8) or missing values. The 35 former placeholders are still listed in `education_attendance_flag`, so the original text is not lost. Code 3 (“do not know”) is a non-response value, not a gap, and is handled later alongside the other special codes.

2.4 Filling gaps using the other datasets

Age in D01

The data dictionary gives a valid age range of 0–110, but D01 has 90 missing ages and 18 ages outside that range. D02 and D03 also record age for the same people, so instead of deleting these rows, we check whether the other datasets can supply the correct value. D02 and D03 are compared with each other, and D01 is compared with D03 wherever D01’s age is valid:

```
age_check <- D01 |>
  left_join(
    select(D03, household_id, person_number, age_d03 = age),
    by = c("household_id", "person_number")
  ) |>
  left_join(
    select(D02, household_id, person_number, age_d02 = age),
    by = c("household_id", "person_number")
  )

age_check |>
  summarise(
    rows = n(),
    d02_vs_d03_mismatches = sum(age_d02 != age_d03),
    d03_ages_outside_0_110 = sum(age_d03 < 0 | age_d03 > 110),
    d01_valid_ages_agreeing_with_d03 = sum(
      age == age_d03 & between(age, 0, 110),
      na.rm = TRUE
    )
  )
```

```

),
d01_valid_ages_disagreeing_with_d03 = sum(
  age != age_d03 & between(age, 0, 110),
  na.rm = TRUE
),
d01_missing = sum(is.na(age)),
d01_invalid = sum(age < 0 | age > 110, na.rm = TRUE)
)

```

	d02_vs_d03_matches	d03_ages_outside_0_110	d01_valid_ages_agreeing_with_d03	d01_valid_ages_disagreeing_with_d03	d01_missing	d01_invalid
rows						
70440	0	0	70332	0	90	18

```

age_check |>
  filter(age < 0 | age > 110) |>
  select(household_id, person_number, age_d01 = age, age_d02, age_d03)

```

household_id	person_number	age_d01	age_d02	age_d03
199105830000011401	05	-7	17	17
199158110000016001	03	175	27	27
260102290000001701	06	-3	5	5
260108810000013901	05	150	17	17
260112230000001401	01	150	47	47
299121450000007401	02	-7	52	52
373100110000003801	03	-7	34	34
379100430000002501	02	-7	49	49
467104490000006001	05	-7	20	20
591100150000026201	01	150	35	35
662107920000006401	01	175	18	18
662110160000001401	03	150	14	14
760111690000006401	10	-3	0	0
798119520000031801	02	175	42	42
876104290000008301	02	150	47	47
877100190000010401	07	150	13	13
960101520000008501	02	-3	16	16
96310259000002301	02	-3	14	14

D02 and D03 agree on every person and contain no invalid ages, and D01 agrees with them on every valid age. The only disagreements are D01's invalid values, so D03's age can be trusted

as a replacement. Deleting these people would throw away valid education and employment records for no good reason.

D01's missing and invalid ages are therefore replaced with D03's age, and `age_flag` notes which rows were changed and why (including the original value for invalid ages):

```
D01 <- age_check |>
  mutate(

    age_flag = case_when(
      is.na(age) ~ "missing - filled from D03",
      age < 0 | age > 110 ~ paste0("invalid (", age, ") - replaced from D03")
    ),

    age = if_else(
      is.na(age) | age < 0 | age > 110, age_d03, age)
  ) |>
  select(-age_d02, -age_d03)
```

The check below confirms the result and counts how many ages were filled in or replaced:

```
tibble(
  rows = nrow(D01),

  missing_age = sum(is.na(D01$age)),

  min_age = min(D01$age),

  max_age = max(D01$age),

  ages_differing_from_d03 = sum(D01$age != age_check$age_d03)
)
```

rows	missing_age	min_age	max_age	ages_differing_from_d03
70440	0	0	110	0

```
D01 |>
  count(
    age_flag = if_else(
      is.na(age_flag),
      "unchanged",
```

```

    str_remove(age_flag, " \\(.*\\)")
  )
)

```

age_flag	n
invalid - replaced from D03	18
missing - filled from D03	90
unchanged	70332

108 ages are repaired and none are deleted. D01's age now lies between 0 and 110 with no missing values, and matches D02 and D03 for every person.

Education level in D02

education_level_code in D02 has 120 missing values. D03 records the same variable for the same people, so the two are compared to see whether D03 can fill the gaps:

```

D02_education_missing_before <- sum(is.na(D02$education_level_code)) #Kept for the before/after
education_check <- D02 |>
  left_join(
    select(D03, household_id, person_number, education_d03 = education_level_code),
    by = c("household_id", "person_number")
  )

education_check |>
  summarise(
    rows = n(),
    missing_in_d02 = sum(is.na(education_level_code)),
    missing_in_d03 = sum(is.na(education_d03)),
    agree_where_both_present =
      sum(education_level_code == education_d03, na.rm = TRUE),
    disagree_where_both_present =
      sum(education_level_code != education_d03, na.rm = TRUE),
    d02_missing_but_d03_present =
      sum(is.na(education_level_code) & !is.na(education_d03))
  )

```

rows	miss- ing_in_d02	miss- ing_in_d03	dis- agree_where_both_present	dis- agree_where_both_present	d02_miss- but_d03_present
70440	120	0	70320	0	120

D02 and D03 never disagree where both have a value, and D03 has no missing values, so D02's gaps are filled from D03, with a note in `education_level_flag` on each filled row:

```
D02 <- education_check |>
  mutate(
    education_level_flag = if_else(is.na(education_level_code), "missing - filled from D03",
    education_level_code = coalesce(education_level_code, education_d03)
  ) |>
  select(-education_d03)

tibble(

  missing_before = D02_education_missing_before,

  missing_after = sum(is.na(D02$education_level_code)),

  rows_flagged = sum(!is.na(D02$education_level_flag))
)
```

missing_before	missing_after	rows_flagged
120	0	120

D02's education level is now complete. Code 99 appears in both D02 and D03 but is not in the dictionary. It is a recorded value rather than a gap, so it cannot be filled from D03, and its meaning stays unknown like the other special codes.

3. Column Selection

The four datasets contain 83 columns, but most of them either do not measure part of the research pathway, repeat information from another dataset, or give away the outcome. This section keeps only the columns the analysis needs. The full reasoning for each column is in the Task 1 write-up (Data Selection and Analytical Grain), and the checks below support the main decisions to drop columns.

Columns that add no information or repeat another column

The first check looks at `survey_year`, which holds a single value, and at the messy `province` text column, whose different spellings reduce to the same provinces as `province_code` once they are trimmed and the capitalisation is fixed:

```
tibble(  
  check = c(  
    "distinct survey_year values in D01",  
    "distinct survey_year values in D02",  
    "distinct survey_year values in D03",  
    "distinct survey_year values in D08",  
    "distinct province spellings in D01",  
    "distinct province spellings after trimming and case-fixing",  
    "distinct province_code values"  
  ),  
  
  result = c(  
    n_distinct(D01$survey_year),  
    n_distinct(D02$survey_year),  
    n_distinct(D03$survey_year),  
    n_distinct(D08$survey_year),  
    n_distinct(D01$province),  
    n_distinct(str_to_title(str_squish(D01$province))),  
    n_distinct(D01$province_code)  
  )  
)
```

check	result
distinct survey_year values in D01	1
distinct survey_year values in D02	1
distinct survey_year values in D03	1
distinct survey_year values in D08	1
distinct province spellings in D01	24
distinct province spellings after trimming and case-fixing	9
distinct province_code values	9

```
D01 |> distinct(  
  province_code,  
  province_clean = str_to_title(str_squish(province))  
) |> arrange(province_code)
```

province_code	province_clean
1	Western Cape
2	Eastern Cape
3	Northern Cape
4	Free State
5	Kwazulu-Natal
6	North West
7	Gauteng
8	Mpumalanga
9	Limpopo

The next check compares the columns that appear in more than one dataset. If they never differ for the same person or household, only one copy is needed. It also checks whether `age_group_code` can be rebuilt from `age`:

```
persons <- D01 |>
  inner_join(
    D02,
    by = c("household_id", "person_number"),
    suffix = c("", "_d02")
  ) |>
  inner_join(
    D03,
    by = c("household_id", "person_number"),
    suffix = c("", "_d03")
  )

tibble(

  column = c("sex_code", "province_code", "geo_type_code", "person_weight", "age"),

  people_compared = nrow(persons),

  mismatches_D01_vs_D02 = c(
    sum(persons$sex_code != persons$sex_code_d02),
    sum(persons$province_code != persons$province_code_d02),
    sum(persons$geo_type_code != persons$geo_type_code_d02),
    sum(persons$person_weight != persons$person_weight_d02),
    sum(persons$age != persons$age_d02)
  ),
```

```

mismatches_D01_vs_D03 = c(
  sum(persons$sex_code != persons$sex_code_d03),
  sum(persons$province_code != persons$province_code_d03),
  sum(persons$geo_type_code != persons$geo_type_code_d03),
  sum(persons$person_weight != persons$person_weight_d03),
  sum(persons$age != persons$age_d03)
)
)

```

column	people_compared	mis- matches_D01_vs_D02	mis- matches_D01_vs_D03
sex_code	70440	0	0
province_code	70440	0	0
geo_type_code	70440	0	0
person_weight	70440	0	0
age	70440	0	0

```

#D08 against D01 at household level
D01 |>
  distinct(household_id, province_code, geo_type_code) |>
  inner_join(
    D08,
    by = "household_id",
    suffix = c("", "_d08")
  ) |>
  summarise(
    households_compared = n(),
    province_mismatch = sum(province_code != province_code_d08),
    geo_type_mismatch = sum(geo_type_code != geo_type_code_d08)
  )

```

households_compared	province_mismatch	geo_type_mismatch
20940	0	0

```

tibble(
  age_group_matches_age =
    sum(as.integer(

```

```

    cut(D01$age, c(-Inf, seq(4, 74, 5), Inf))
  ) == D01$age_group_code),

  rows = nrow(D01)

)

```

age_group_matches_age	rows
70440	70440

Columns that give away the employment outcome

These columns are cross-tabulated against `employment_status2code`. If a column's values line up with employment status, it is part of how the outcome is defined rather than something that explains it, and keeping it would leak the answer into the model:

```
D03 |> count(employment_status1code, employment_status2code)
```

employment_status1code	employment_status2code	n
1	1	19005
2	2	10166
8	8	19776
9	3	21393
NA	1	25
NA	2	19
NA	3	29
NA	8	27

```
D03 |> count(wage_work_code, employment_status2code)
```

wage_work_code	employment_status2code	n
1	1	15918
2	1	3112
2	2	10185
2	3	21420
8	8	19803

wage_work_code	employment_status2code	n
9	3	2

```
D03 |> count(looking_for_work_code, employment_status2code)
```

looking_for_work_code	employment_status2code	n
1	2	10185
1	3	206
2	3	21214
8	1	19030
8	3	2
8	8	19803

```
D03 |> count(sector_code, employment_status2code)
```

sector_code	employment_status2code	n
1	1	14343
2	1	4368
3	1	212
8	1	107
8	2	10185
8	3	21422
8	8	19803

Columns that cannot be used or are not needed

`salary_amount_raw` is checked to see whether it holds usable Rand amounts, `grade_code` is compared with `education_level_code` to see whether it adds anything new, and the schooling detail columns are checked for how often they are “not applicable” among 15–64-year-olds:

```
D03 |> count(salary_amount_raw)
```

salary_amount_raw	n
1e+00	7456
2e+00	9217

salary_amount_raw	n
3e+00	2243
8e+00	51504
1e+08	13
1e+09	4
1e+10	3

```
D02 |>
  filter(
    between(age, 15, 64),
    education_attendance_code == 1,
    grade_code %in% 1:12,
    education_level_code %in% 0:12
  ) |>
  count(grade_minus_education_level = grade_code - education_level_code)
```

grade_minus_education_level	n
1	5611

```
D02 |>
  filter(between(age, 15, 64)) |>
  summarise(
    across(
      c(
        public_private_institution_code,
        transport_mode_to_education_code,
        travel_time_to_education,
        bursary_code,
        total_education_fees,
        no_fees_school_code,
        school_food_programme_code,
        absence_code, absence_days
      ), ~ round(mean(.x %in% c(8, 88)) * 100, 1)
    )
  ) |>
  pivot_longer(
    everything(),
    names_to = "column",
    values_to = "percent_not_applicable_age_15_64"
  )
```

column	percent_not_applicable_age_15_64
public_private_institution_code	83.7
transport_mode_to_education_code	86.8
travel_time_to_education	85.5
bursary_code	83.7
total_education_fees	84.0
no_fees_school_code	91.0
school_food_programme_code	87.6
absence_code	87.6
absence_days	99.3

Education columns for Link A (digital access → education)

For young people aged 15–24, these tables show which education columns are useful for the link between digital access and education. They look at the type of institution attended, the reasons for not attending, and whether using the internet for study simply mirrors attendance:

```
youth <- D02 |> filter(between(age, 15, 24))

youth |>
  filter(education_attendance_code == 1) |>
  count(institution_type_code)
```

institution_type_code	n
2	5600
3	4
4	1
5	507
6	306
7	151
8	13
9	67

```
youth |>
  filter(education_attendance_code == 2) |>
  count(reason_not_attending_code, sort = TRUE) #Kept for descriptive use only
```

reason_not_attending_code	n
5	1647
2	785
6	649
18	635
9	464
16	387
7	245
8	159
1	115
12	98
11	89
10	71
14	58
4	24
17	23
3	19
15	15
13	1

```

youth |>
  left_join(
    select(
      D08,
      household_id,
      study_internet_code
    ),
    by = "household_id"
  ) |>
  filter(
    education_attendance_code %in% 1:2,
    study_internet_code %in% 1:2
  ) |>
  count(study_internet_code, education_attendance_code)

```

study_internet_code	education_attendance_code	n
1	1	633
1	2	156
2	1	6016
2	2	5328

Keeping the selected columns

Only the columns selected in the Task 1 write-up are kept, together with the join keys and the `_flag` columns created during cleaning, which are still needed for the checks later on. The column names and types are stored first, so that the dropped columns can be listed below and the final types compared later.

```
#Column names before selection, kept to show which columns were dropped
D01_columns_before <- names(D01)
D02_columns_before <- names(D02)
D03_columns_before <- names(D03)
D08_columns_before <- names(D08)

D01 <- D01 |>
  select(
    household_id,
    person_number,
    psu,
    province_code,
    sex_code,
    age,
    population_group_code,
    geo_type_code,
    metro_indicator,
    person_weight,
    household_id_flag,
    age_flag
  )

D02 <- D02 |>
  select(
    household_id,
    person_number,
    education_level_code,
    education_attendance_code,
    institution_type_code,
    reason_not_attending_code,
    education_level_flag,
    education_attendance_flag
  )

D03 <- D03 |>
  select(
```

```

    household_id,
    person_number,
    employment_status2code,
    lab_salary_raw
  )

D08 <- D08 |>
  select(
    household_id,
    fixed_internet_code,
    mobile_internet_code,
    computer_asset_code,
    household_smartphones
  )

#Column types after selection, compared with the final types later
D01_types_before <- sapply(D01, class)
D02_types_before <- sapply(D02, class)
D03_types_before <- sapply(D03, class)
D08_types_before <- sapply(D08, class)

```

The table below compares the number of columns at each stage:

```

tibble(

  dataset = c("D01", "D02", "D03", "D08"),

  columns_in_uncleaned_data = uncleaned_summary$cols,

  columns_before_selection = c(
    length(D01_columns_before),
    length(D02_columns_before),
    length(D03_columns_before),
    length(D08_columns_before)
  ),

  columns_after_selection = c(
    ncol(D01),
    ncol(D02),
    ncol(D03),
    ncol(D08)
  )
)

```

```
)
)
```

dataset	columns_in_un- cleaned_data	columns_before_selection	columns_after_selection
D01	17	19	12
D02	23	25	8
D03	25	25	4
D08	18	18	5

Every dropped column, grouped by dataset and the reason it was dropped:

```
#Dropped columns for each dataset, stacked into one table
columns_dropped <- bind_rows(
  tibble(
    dataset = "D01",
    column = setdiff(D01_columns_before, names(D01))
  ),
  tibble(
    dataset = "D02",
    column = setdiff(D02_columns_before, names(D02))
  ),
  tibble(
    dataset = "D03",
    column = setdiff(D03_columns_before, names(D03))
  ),
  tibble(
    dataset = "D08",
    column = setdiff(D08_columns_before, names(D08))
  )
) |>

#Detailed Table explainning why each column gets dropped
mutate(
  reason = case_when(
    column == "survey_year" ~ "No information (always 2024)",
```

```

column == "province" ~ "Messy text, rebuilt from province_code",

column %in% c(
  "province_code",
  "age",
  "sex_code",
  "geo_type_code",
  "person_weight"
) ~ "Duplicate of D01",

column == "education_level_code" ~ "Duplicate of D02 (used to fill D02's gaps in 2.4)"

column %in% c(
  "age_group_code",
  "employment_status1code",
  "grade_code"
) ~ "Duplicate of another column",

column %in% c(
  "wage_work_code",
  "business_work_code",
  "volunteer_work_code",
  "retired_code",
  "looking_for_work_code",
  "could_start_work_code",
  "transport_to_work_code",
  "minutes_to_work",
  "sector_code",
  "work_internet_code",
  "study_internet_code"
) ~ "Leakage: reveals or defines an outcome",

column %in% c(
  "language_code",
  "relationship_to_head_code",
  "marital_status_code"
) ~ "Not needed for any hypothesis",

column == "metro_code" ~ "Too detailed (disclosure risk)",

column %in% c(
  "public_private_institution_code",

```

```

    "transport_mode_to_education_code",
    "travel_time_to_education",
    "bursary_code",
    "total_education_fees",
    "no_fees_school_code",
    "school_food_programme_code",
    "absence_code",
    "absence_days"
  ) ~ "Only recorded for people already attending",

column == "salary_amount_raw" ~ "Unusable (not Rand amounts)",

column %in% c(
  "salary_period_code",
  "salary_category_code"
) ~ "Salary detail not needed (lab_salary_raw kept)",

column %in% c(
  "telephone_code",
  "cellphone_code",
  "household_mobile_phones"
) ~ "Phones, not internet access",

column %in% c(
  "public_wi-fi_internet_code",
  "library_internet_code",
  "cafe_internet_code"
) ~ "Internet access outside the home",

column == "household_weight" ~ "Household weight, not needed at person grain",

TRUE ~ "NOT CLASSIFIED"
)
)

columns_dropped |>
  group_by(dataset, reason) |>
  summarise(
    columns_dropped = n(),
    columns = paste(column, collapse = ", "),
    .groups = "drop"
  )

```

dataset	reason	columns	columns collapsed
D01	Duplicate of another column	1	age_group_code
D01	Messy text, rebuilt from province_code	1	province
D01	No information (always 2024)	1	survey_year
D01	Not needed for any hypothesis	3	language_code, relationship_to_head_code, marital_status_code
D01	Too detailed (disclosure risk)	1	metro_code
D02	Duplicate of D01	5	province_code, age, sex_code, geo_type_code, person_weight
D02	Duplicate of another column	1	grade_code
D02	Messy text, rebuilt from province_code	1	province
D02	No information (always 2024)	1	survey_year
D02	Only recorded for people already attending	9	public_private_institution_code, transport_mode_to_education_code, travel_time_to_education, bursary_code, total_education_fees, no_fees_school_code, school_food_programme_code, absence_code, absence_days
D03	Duplicate of D01	5	province_code, age, sex_code, geo_type_code, person_weight
D03	Duplicate of D02 (used to fill D02's gaps in 2.4)	1	education_level_code
D03	Duplicate of another column	1	employment_status1code
D03	Leakage: reveals or defines an outcome	9	wage_work_code, business_work_code, volunteer_work_code, retired_code, transport_to_work_code, minutes_to_work, sector_code, looking_for_work_code, could_start_work_code
D03	Messy text, rebuilt from province_code	1	province

dataset	reason	columns	dropped
D03	No information (always 2024)	1	survey__year
D03	Salary detail not needed (lab_salary_raw kept)	2	salary__period__code, salary__category__code
D03	Unusable (not Rand amounts)	1	salary__amount__raw
D08	Duplicate of D01	2	province__code, geo__type__code
D08	Household weight, not needed at person grain	1	household__weight
D08	Internet access outside the home	3	public_wi-fi__internet__code, library__internet__code, cafe__internet__code
D08	Leakage: reveals or defines an outcome	2	study__internet__code, work__internet__code
D08	Messy text, rebuilt from province__code	1	province
D08	No information (always 2024)	1	survey__year
D08	Phones, not internet access	3	telephone__code, cellphone__code, household__mobile__phones

The four datasets go from 83 columns to 29, of which 4 are the `_flag` columns from section 2. All 58 dropped columns have a documented reason (0 are marked “NOT CLASSIFIED”). `select()` only removes columns, so every person and household cleaned in section 2 is still there.

4. Special Values

Some of the kept columns contain values that are not ordinary answers: “not applicable” codes, non-response codes, codes that are missing from the data dictionary, and placeholder amounts in `lab_salary_raw`. Each of these is looked at and classified before anything is changed.

4.1 Classifying the special codes

The kept code columns mix valid categories with these special codes. Treating them all as normal categories would put “do not know” alongside real answers, while deleting the rows would throw away the same people’s other valid answers. The counts below show which special codes appear in each column:

```
D02 |>
  count(education_level_code, sort = TRUE) |>
  filter(education_level_code %in% c(98, 99, 29, 28))
```

education_level_code	n
98	9392
29	661
28	278
99	53

```
D02 |> count(education_attendance_code)
```

education_attendance_code	n
1	20590
2	43540
3	20
8	6255
NA	35

```
D02 |> count(institution_type_code == 88, name = "rows")
```

institution_type_code == 88	rows
FALSE	20599
TRUE	49841

```
D02 |> count(reason_not_attending_code == 88, name = "rows")
```

reason_not_attending_code == 88	rows
FALSE	43562
TRUE	26878

```
D08 |> count(fixed_internet_code)
```

fixed_internet_code	n
1	2921
2	18017
9	2

```
D08 |> count(mobile_internet_code)
```

mobile_internet_code	n
1	15385
2	5493
9	2
NA	60

```
D08 |> count(computer_asset_code)
```

computer_asset_code	n
1	4523
2	16414
9	3

Each special code is classified as a valid category, a “Not applicable” level or **NA**, as summarised in the table below. Nothing is changed yet; the classification below is only applied later, when the label columns are created, because any code left out of **factor()**’s levels becomes **NA**.

- Codes 98 (No schooling) and 28 (Other) in **education_level_code** are real education levels, so they stay as valid categories.
- Codes 29 and 3 mean “do not know”, and code 9 in **computer_asset_code** means “Unspecified”. These are non-responses rather than categories, so they become **NA**.
- Code 99 in **education_level_code** and code 9 in **fixed_internet_code** and **mobile_internet_code** are not in the data dictionary. Their meaning is unknown, so they become **NA**.

- Code 88 in `institution_type_code` and `reason_not_attending_code` means the question did not apply to that person (for example, only people who are not attending are asked why). It becomes a “Not applicable” level rather than a missing value.
- Attendance code 8 only occurs under age 15, so it disappears once the working-age filter is applied.

```
tibble(

  column = c(
    "education_level_code",
    "education_level_code",
    "education_level_code",
    "education_level_code",
    "education_attendance_code",
    "education_attendance_code",
    "institution_type_code",
    "reason_not_attending_code",
    "fixed_internet_code",
    "mobile_internet_code",
    "computer_asset_code"
  ),

  code = c(98, 28, 29, 99, 3, 8, 88, 88, 9, 9, 9),

  meaning = c(
    "No schooling",
    "Other",
    "Do not know",
    "Not in the data dictionary",
    "Do not know",
    "Not applicable (only under age 15)",
    "Not applicable",
    "Not applicable",
    "Not in the data dictionary",
    "Not in the data dictionary",
    "Unspecified"
  ),

  classification = c(
    "Valid category",
    "Valid category",
    "NA (non-response)",
    "NA (undocumented code)",
```

```

"NA (non-response)",
"Removed by record selection",
"'Not applicable' level",
"'Not applicable' level",
"NA (undocumented code)",
"NA (undocumented code)",
"NA (non-response)"
),

rows = c(
  sum(D02$education_level_code %in% 98),
  sum(D02$education_level_code %in% 28),
  sum(D02$education_level_code %in% 29),
  sum(D02$education_level_code %in% 99),
  sum(D02$education_attendance_code %in% 3),
  sum(D02$education_attendance_code %in% 8),
  sum(D02$institution_type_code %in% 88),
  sum(D02$reason_not_attending_code %in% 88),
  sum(D08$fixed_internet_code %in% 9),
  sum(D08$mobile_internet_code %in% 9),
  sum(D08$computer_asset_code %in% 9)
)
)

```

column	code	meaning	classification	rows
education_level_code	98	No schooling	Valid category	9392
education_level_code	28	Other	Valid category	278
education_level_code	29	Do not know	NA (non-response)	661
education_level_code	99	Not in the data dictionary	NA (undocumented code)	53
education_atten- dance_code	3	Do not know	NA (non-response)	20
education_atten- dance_code	8	Not applicable (only under age 15)	Removed by record selection	6255
institu- tion_type_code	88	Not applicable	'Not applicable' level	49841
reason_not_attend- ing_code	88	Not applicable	'Not applicable' level	26878
fixed_internet_code	9	Not in the data dictionary	NA (undocumented code)	2

column	code	meaning	classification	rows
mobile_inter- net_code	9	Not in the data dictionary	NA (undocumented code)	2
computer_as- set_code	9	Unspecified	NA (non-response)	3

Two groups of values were already NA before this section. They stay NA, because there is no dictionary code to give them:

- The 35 missing values in `education_attendance_code` were not gaps in the original data, because the uncleaned column had no missing values. They are the text placeholders (“N/A”, “Unknown”, “not recorded”) that were set to missing in 2.3, with the original text kept in `education_attendance_flag`.
- The 60 missing values in `mobile_internet_code` were already missing in the uncleaned data.

No values are changed in this section, and every `_code` column keeps its original codes. The classification only takes effect when the label columns are created, where the table counting codes without a label confirms it: after record selection, the rows left without a label are exactly the rows holding the codes classified as NA here.

4.2 Placeholder values in salary

`lab_salary_raw` mixes real Rand amounts with two placeholder codes, 888888888 and 999999999. If they were left in, they would be read as salaries of almost R1 billion. Before deciding what to do with them, the placeholders are compared with employment status to see what each one actually means:

```
D03 |>
  mutate(
    salary_type = case_when(
      lab_salary_raw == 888888888 ~ "888888888 (don't know/refused)",
      lab_salary_raw == 999999999 ~ "999999999 (not applicable)",
      TRUE ~ "Real salary value"
    )
  ) |>
count(employment_status2code, salary_type) |>
pivot_wider(
  names_from = employment_status2code,
  values_from = n,
```

```
values_fill = 0
)
```

salary__type	1	2	3	8
888888888 (don't know/refused)	107	10185	21422	19803
999999999 (not applicable)	612	0	0	0
Real salary value	18311	0	0	0

The cross-table shows that 888888888 is used as “not applicable” for everyone without a job, while 999999999 is non-response among the employed. Both placeholders become NA in a new `salary` column. A second new column, `salary_status`, records why a salary is missing, based on the value and `employment_status2code` together rather than the placeholder alone. `lab_salary_raw` itself stays unchanged.

```
D03_salary_missing_before <- sum(D03$lab_salary_raw %in% c(888888888, 999999999))

D03 <- D03 |>
  mutate(
    salary = if_else(
      lab_salary_raw %in% c(888888888, 999999999),
      NA_real_,
      lab_salary_raw
    ),
    salary_status = case_when(
      !is.na(salary) ~ "Reported",
      lab_salary_raw == 999999999 ~ "Non-response (employed)",
      lab_salary_raw == 888888888 & employment_status2code == 1
      ~ "Non-response (employed)",
      lab_salary_raw == 888888888 ~ "Not applicable (not employed)",
      TRUE ~ NA_character_
    )
  )
```

18,311 of 19,030 employed people report a real salary (96%). This corrects Milestone 1’s “74% unusable” figure, which included people who could never have a salary in the first place. The check below confirms that no placeholder values are left in `salary`:

```
D03 |> count(salary_status, has_salary = !is.na(salary))
```

salary_status	has_salary	n
Non-response (employed)	FALSE	719
Not applicable (not employed)	FALSE	51410
Reported	TRUE	18311

```
tibble(
  placeholders_before = D03_salary_missing_before,
  placeholders_remaining_in_salary =
    sum(D03$salary %in% c(888888888, 999999999), na.rm = TRUE)
)
```

placeholders_before	placeholders_remaining_in_salary
52129	0

5. Record Selection (ages 15–64)

The analysis focuses on the working-age population, people aged 15–64. D01 is filtered on `between(age, 15, 64)`, and D02 and D03 then keep only the matching people using `semi_join()`. All D08 households are kept; the households with no working-age member are reported once the datasets are joined.

Removing children and people aged 65+ also removes information that later steps still need: household-level features built from the full household composition, and a smartphone-per-person check that needs the full household size, not just the working-age members. So before the filter runs, every member of each household is counted:

```
#Counts all members of each household before the 15-64 filter removes children and people aged 15-64
household_composition <- D01 |>
  group_by(household_id) |>
  summarise(
    household_size = n(),
    n_children_under15 = sum(age < 15),
    n_elderly_65plus = sum(age > 64),
    .groups = "drop"
  )

tibble(
```

```

households = nrow(household_composition),

people_counted = sum(household_composition$household_size),

people_in_D01 = nrow(D01),

children_counted = sum(household_composition$n_children_under15),

aged_65_plus_counted = sum(household_composition$n_elderly_65plus)
)

```

households	people_counted	people_in_D01	children_counted	aged_65_plus_counted
20940	70440	70440	19803	5224

The age filter is then applied, and the full household size is kept aside for the outlier check later:

```

D01_rows_before_selection <- nrow(D01)
D01_under_15 <- sum(D01$age < 15)
D01_65_plus <- sum(D01$age > 64)

household_size <- D01 |>
  count(household_id, name = "household_size")

D01 <- D01 |> filter(between(age, 15, 64))

D02 <- semi_join(D02, D01, by = c("household_id", "person_number"))

D03 <- semi_join(D03, D01, by = c("household_id", "person_number"))

```

The table below confirms that all three person datasets were filtered to the same people:

```

tibble(

  dataset = c("D01", "D02", "D03"),

  rows_before = D01_rows_before_selection,

  rows_after = c(

```

```

nrow(D01),
nrow(D02),
nrow(D03)
),

removed_under_15 = D01_under_15,

removed_65_plus = D01_65_plus,

distinct_households = c(
  n_distinct(D01$household_id),
  n_distinct(D02$household_id),
  n_distinct(D03$household_id)
),

identical_keys_to_D01 = c(
  TRUE,
  setequal(
    paste(D01$household_id, D01$person_number),
    paste(D02$household_id, D02$person_number)
  ),
  setequal(
    paste(D01$household_id, D01$person_number),
    paste(D03$household_id, D03$person_number)
  )
)
)

```

dataset	rows_before	rows_after	re-moved_under_15	re-moved_65_plus	distinct_households	identical_keys_to_D01
D01	70440	45413	19803	5224	19780	TRUE
D02	70440	45413	19803	5224	19780	TRUE
D03	70440	45413	19803	5224	19780	TRUE

All three person datasets now have 45,413 rows and identical keys, covering 19,780 households. 19,803 people under 15 and 5,224 people aged 65+ were removed.

6. Data Transformation: Recoding Codes into Labelled Factors

The `_code` columns are numbers that stand for categories, which makes them risky to use as they are. A model would treat province 9 as “more” than province 1, and someone reading a report cannot tell what code 5 means without the data dictionary.

For every kept `_code` column, a label column with the same name (without the suffix) is added using `factor(..., levels =, labels =)` and the labels from the data dictionary. Any code left out of `levels =` becomes NA, which is how the classification from section 4 is applied. The `_code` columns themselves are never changed, so every label can be traced back to its original code.

```
D01 <- D01 |>
  mutate(
    province =
      factor(
        province_code,
        levels = 1:9,
        labels = c(
          "Western Cape",
          "Eastern Cape",
          "Northern Cape",
          "Free State",
          "KwaZulu-Natal",
          "North West",
          "Gauteng",
          "Mpumalanga",
          "Limpopo")
      ),

    sex = factor(
      sex_code,
      levels = 1:2,
      labels = c("Male", "Female")
    ),

    population_group = factor(
      population_group_code,
      levels = 1:4,
      labels = c(
        "Black African",
        "Coloured",
        "Indian/Asian",
```

```

    "White")
  ),

  geo_type = factor(
    geo_type_code,
    levels = 1:3,
    labels = c(
      "Urban",
      "Traditional",
      "Farms")
  ),

  metro = factor(
    metro_indicator,
    levels = 1:2,
    labels = c(
      "Metro",
      "Non-metro")
  )
)

```

```

D02 <- D02 |>
mutate(
  education_level =
    factor(
      education_level_code,
      levels = c(98, 0:27, 28),
      labels = c(
        "No schooling",
        "Grade R/0", "Grade 1/Sub A/Class 1", "Grade 2/Sub B/Class 2",
        "Grade 3/Standard 1/ABET/AET 1", "Grade 4/Standard 2",
        "Grade 5/Standard 3/ABET/AET 2", "Grade 6/Standard 4",
        "Grade 7/Standard 5/ABET/AET 3", "Grade 8/Standard 6/Form 1",
        "Grade 9/Standard 7/Form 2/ABET/AET 4/NCV 1",
        "Grade 10/Standard 8/Form 3/NCV 2",
        "Grade 11/Standard 9/Form 4/NCV 3",
        "Grade 12/Standard 10/Form 5/NSC/Matric/NCV 4",
        "NTC I/N1/NQF 1", "NTC II/N2/NQF 2", "NTC III/N3/NQF 3",
        "N4/NTC 4/Occupational Cert NQF 5", "N5/NTC 5/Occupational Cert NQF 5",
        "N6/NTC 6/Occupational Cert NQF 5",
        "Certificate less than Grade 12", "Diploma less than Grade 12",

```

```

    "Higher/National/Advanced Certificate with Grade 12/NQF 5",
    "Diploma with Grade 12/Occupational Cert NQF 6",
    "Higher Diploma/B-Tech/Occupational Cert NQF 7",
    "Bachelor's Degree/Occupational Cert NQF 7",
    "Honours Degree/Postgrad Diploma/Occupational Cert NQF 8",
    "Post Higher Diploma (M-Tech/Master's) NQF 9",
    "Doctoral Degree (D-Tech/PhD) NQF 10",
    "Other"
  )
),
education_attendance = factor(
  education_attendance_code,
  levels = 1:2,
  labels = c("Yes", "No")
),
institution_type = factor(
  institution_type_code,
  levels = c(1:9, 88),
  labels = c(
    "Pre-school (incl. ECD centre)",
    "School (Grade R-12)",
    "ABET/AET centre",
    "Literacy classes",
    "Higher education institution (university / university of technology)",
    "TVET college",
    "Other college",
    "Home-based education / home schooling",
    "Other educational institution",
    "Not applicable"
  )
),
reason_not_attending = factor(
  reason_not_attending_code,
  levels = c(1:18, 88),
  labels = c(
    "Too old/young",
    "Completed education / satisfied / doesn't want to study",
    "Institution too far",
    "Difficulty getting there (transport)",
    "No money for fees",
    "Working (at home, business or job)",
    "Family commitment (e.g. child minding)",
    "Education useless / not interesting",

```

```

    "Unable to perform at school",
    "Illness",
    "Pregnancy",
    "Failed exams",
    "Got married",
    "Disability",
    "Violence in school",
    "Not accepted for enrolment",
    "No time / too busy",
    "Other reason",
    "Not applicable"
  )
)
)

```

```

D03 <- D03 |>
  mutate(
    employment_status = factor(
      employment_status2code,
      levels = 1:3,
      labels = c(
        "Employed",
        "Unemployed",
        "Not economically active"
      )
    )
  )
)

```

```

D08 <- D08 |>
  mutate(
    fixed_internet = factor(
      fixed_internet_code,
      levels = 1:2,
      labels = c("Yes", "No")
    ),
    mobile_internet = factor(
      mobile_internet_code,
      levels = 1:2,
      labels = c("Yes", "No")
    ),
    computer_asset = factor(
      computer_asset_code,

```

```

levels = 1:2,
labels = c("Yes", "No")
)
)

```

The counts below show that every code maps to exactly one label, and that only the codes classified as non-response or undocumented in section 4 are left without one:

```
D01 |> count(province_code, province)
```

province_code	province	n
1	Western Cape	4484
2	Eastern Cape	5584
3	Northern Cape	1920
4	Free State	2603
5	KwaZulu-Natal	7989
6	North West	2665
7	Gauteng	11612
8	Mpumalanga	3675
9	Limpopo	4881

```
D01 |> count(sex_code, sex)
```

sex_code	sex	n
1	Male	21952
2	Female	23461

```
D01 |> count(population_group_code, population_group)
```

population_group_code	population_group	n
1	Black African	38365
2	Coloured	4177
3	Indian/Asian	816
4	White	2055

```
D01 |> count(geo_type_code, geo_type)
```

geo_type_code	geo_type	n
1	Urban	29330
2	Traditional	14552
3	Farms	1531

```
D01 |> count(metro_indicator, metro)
```

metro_indicator	metro	n
1	Metro	17951
2	Non-metro	27462

```
D02 |> count(education_level_code, education_level)
```

education_level_code	education_level	n
0	Grade R/0	18
1	Grade 1/Sub A/Class 1	111
2	Grade 2/Sub B/Class 2	226
3	Grade 3/Standard 1/ABET/AET 1	291
4	Grade 4/Standard 2	457
5	Grade 5/Standard 3/ABET/AET 2	529
6	Grade 6/Standard 4	888
7	Grade 7/Standard 5/ABET/AET 3	1984
8	Grade 8/Standard 6/Form 1	2648
9	Grade 9/Standard 7/Form 2/ABET/AET 4/NCV 1	4013
10	Grade 10/Standard 8/Form 3/NCV 2	5689
11	Grade 11/Standard 9/Form 4/NCV 3	6548
12	Grade 12/Standard 10/Form 5/NSC/Matric/NCV 4	14633
13	NTC I/N1/NQF 1	32
14	NTC II/N2/NQF 2	65
15	NTC III/N3/NQF 3	135
16	N4/NTC 4/Occupational Cert NQF 5	203
17	N5/NTC 5/Occupational Cert NQF 5	145
18	N6/NTC 6/Occupational Cert NQF 5	289
19	Certificate less than Grade 12	47
20	Diploma less than Grade 12	62

education_level_code	education_level	n
21	Higher/National/Advanced Certificate with Grade 12/NQF 5	503
22	Diploma with Grade 12/Occupational Cert NQF 6	1818
23	Higher Diploma/B-Tech/Occupational Cert NQF 7	377
24	Bachelor's Degree/Occupational Cert NQF 7	1481
25	Honours Degree/Postgrad Diploma/Occupational Cert NQF 8	431
26	Post Higher Diploma (M-Tech/Master's) NQF 9	202
27	Doctoral Degree (D-Tech/PhD) NQF 10	46
28	Other	246
29	NA	499
98	No schooling	782
99	NA	15

```
D02 |> count(education_attendance_code, education_attendance)
```

education_attendance_code	education_attendance	n
1	Yes	7392
2	No	37980
3	NA	18
NA	NA	23

```
D02 |> count(institution_type_code, institution_type)
```

institution_type_code	institution_type	n
2	School (Grade R-12)	5631
3	ABET/AET centre	6
4	Literacy classes	1
5	Higher education institution (university / university of technology)	913
6	TVET college	453
7	Other college	271
8	Home-based education / home schooling	14
9	Other educational institution	105
88	Not applicable	38019

```
D02 |> count(reason_not_attending_code, reason_not_attending)
```

reason_not_attending_code	reason_not_attending	n
1	Too old/young	6943
2	Completed education / satisfied / doesn't want to study	5374
3	Institution too far	50
4	Difficulty getting there (transport)	80
5	No money for fees	7082
6	Working (at home, business or job)	9314
7	Family commitment (e.g. child minding)	1986
8	Education useless / not interesting	519
9	Unable to perform at school	1898
10	Illness	420
11	Pregnancy	212
12	Failed exams	339
13	Got married	38
14	Disability	308
15	Violence in school	20
16	Not accepted for enrolment	547
17	No time / too busy	308
18	Other reason	2563
88	Not applicable	7412

```
D03 |> count(employment_status2code, employment_status)
```

employment_status2code	employment_status	n
1	Employed	18711
2	Unemployed	10135
3	Not economically active	16567

```
D08 |> count(fixed_internet_code, fixed_internet)
```

fixed_internet_code	fixed_internet	n
1	Yes	2921
2	No	18017
9	NA	2

```
D08 |> count(mobile_internet_code, mobile_internet)
```

mobile_internet_code	mobile_internet	n
1	Yes	15385
2	No	5493
9	NA	2
NA	NA	60

```
D08 |> count(computer_asset_code, computer_asset)
```

computer_asset_code	computer_asset	n
1	Yes	4523
2	No	16414
9	NA	3

The last table compares, for each label column, the number of rows holding the codes set to NA in section 4 with the number of rows that have a code but no label:

```
#Rows with a code but no label, compared with the number of rows holding the non-response codes
tibble(
```

```
  label_column = c(
    "education_level",
    "education_attendance",
    "fixed_internet",
    "mobile_internet",
    "computer_asset"
  ),

  codes_set_to_NA = c("29, 99", "3", "9", "9", "9"),

  rows_with_these_codes = c(
    sum(D02$education_level_code %in% c(29, 99)),
    sum(D02$education_attendance_code %in% 3),
    sum(D08$fixed_internet_code %in% 9),
    sum(D08$mobile_internet_code %in% 9),
    sum(D08$computer_asset_code %in% 9)
  ),
```

```

rows_with_codes_but_no_label = c(
  sum(!is.na(D02$education_level_code) & is.na(D02$education_level)),
  sum(!is.na(D02$education_attendance_code) & is.na(D02$education_attendance)),
  sum(!is.na(D08$fixed_internet_code) & is.na(D08$fixed_internet)),
  sum(!is.na(D08$mobile_internet_code) & is.na(D08$mobile_internet)),
  sum(!is.na(D08$computer_asset_code) & is.na(D08$computer_asset))
)
)

```

label_column	codes_set_to_NA	rows_with_these_codes	rows_with_codes_but_no_label
education_level	29, 99	514	514
education_attendance	3	18	18
fixed_internet	9	2	2
mobile_internet	9	2	2
computer_asset	9	3	3

13 label columns are added (5 in D01, 4 in D02, 1 in D03 and 3 in D08), and `no_code` column is changed. The two count columns in the table above match, so the only codes without a label are the non-response and undocumented codes from section 4: 514 education levels, 18 attendance values, 2 fixed internet, 2 mobile internet and 3 computer asset values. `institution_type` and `reason_not_attending` now use the data dictionary’s descriptions, and code 88 is kept as “Not applicable” rather than missing.

The levels of `education_level` follow the dictionary order, which is not a ranking: “No schooling” comes first and “Other” comes last. Any education band built later groups explicit codes rather than relying on this level order.

7. Outlier Review

`age`, `household_smartphones` and `salary` are the only kept columns that are actual measurements rather than categories, so they are the only ones checked for outliers and impossible values. Each is shown as a histogram and a boxplot. The boxplot’s whiskers end at the fences, which sit $1.5 \times$ the interquartile range (IQR) beyond the quartiles, and any point outside the fences is drawn separately as a statistical outlier. Being a statistical outlier does not make a value wrong: it is kept unless there is a clear real-world reason why the value is impossible.

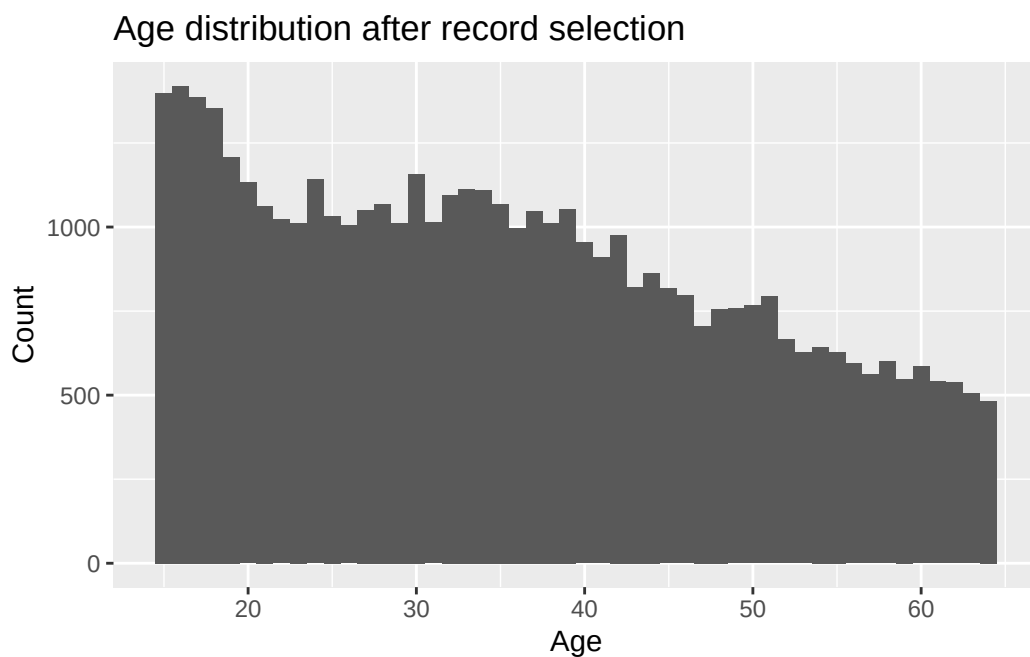
7.1 Age

Extreme ages could distort any analysis by age, so the age distribution is checked after record selection:

```
summary(D01$age)
```

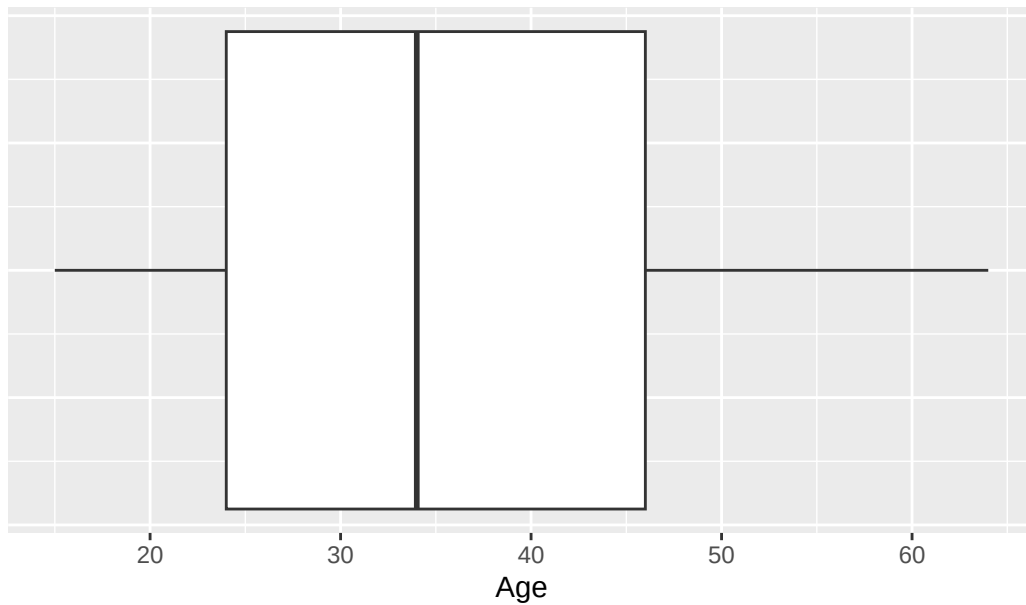
Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
15.00	24.00	34.00	35.71	46.00	64.00

```
ggplot(D01, aes(x = age)) +  
  geom_histogram(binwidth = 1) +  
  labs(title = "Age distribution after record selection", x = "Age", y = "Count")
```



```
ggplot(D01, aes(x = age)) +  
  geom_boxplot() +  
  labs(title = "Age after record selection (boxplot)", x = "Age", y = NULL) +  
  theme(axis.text.y = element_blank(), axis.ticks.y = element_blank())
```

Age after record selection (boxplot)



```
age_boxplot <- tibble(  
  q1 = quantile(D01$age, 0.25),  
  median = median(D01$age),  
  q3 = quantile(D01$age, 0.75),  
  lower_fence = q1 - 1.5 * (q3 - q1),  
  upper_fence = q3 + 1.5 * (q3 - q1),  
  points_below_fence = sum(D01$age < lower_fence),  
  points_above_fence = sum(D01$age > upper_fence)  
)  
age_boxplot
```

q1	median	q3	lower_fence	upper_fence	points_be- low_fence	points_above_fence
24	34	46	-9	79	0	0

No further treatment is needed. Invalid ages were already repaired in 2.4, and record selection (section 5) limits age to 15–64. The summary shows a minimum of 15 and a maximum of 64. The histogram has no isolated values, and the boxplot has 0 points outside its fences (-9 to 79), so there are no age outliers left to deal with.

7.2 Household smartphones

A household cannot own a negative number of smartphones, yet D08 contains 15 negative counts. Very high counts also need a second look, by comparing them with the number of people in the household:

```
summary(D08$household_smartphones)
```

```

      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
-20.00    1.00    2.00    1.76    2.00   13.00

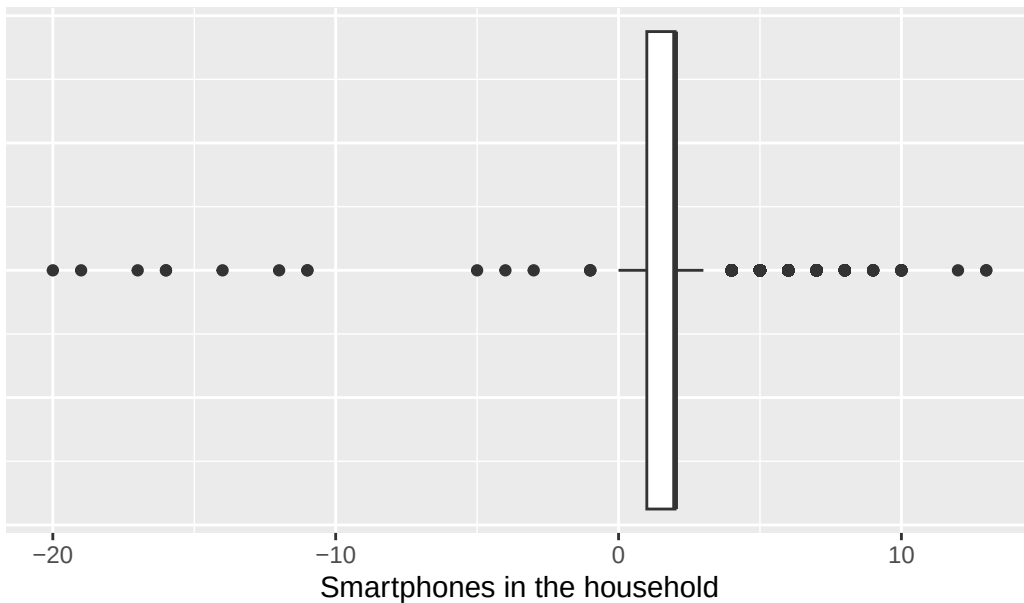
```

```

ggplot(D08, aes(x = household_smartphones)) +
  geom_boxplot() +
  labs(
    title = "Household smartphones before treatment (boxplot)",
    x = "Smartphones in the household",
    y = NULL
  ) +
  theme(axis.text.y = element_blank(), axis.ticks.y = element_blank())

```

Household smartphones before treatment (boxplot)



```
smartphones_boxplot <- tibble(  
  
  q1 = quantile(D08$household_smartphones, 0.25, na.rm = TRUE),  
  median = median(D08$household_smartphones, na.rm = TRUE),  
  q3 = quantile(D08$household_smartphones, 0.75, na.rm = TRUE),  
  lower_fence = q1 - 1.5 * (q3 - q1),  
  upper_fence = q3 + 1.5 * (q3 - q1),  
  points_below_fence = sum(D08$household_smartphones < lower_fence, na.rm = TRUE),  
  points_above_fence = sum(D08$household_smartphones > upper_fence, na.rm = TRUE)  
)  
  
smartphones_boxplot
```

q1	median	q3	lower_fence	upper_fence	points_below_fence	points_above_fence
1	2	2	-0.5	3.5	15	2275

```
D08 |>
  filter(household_smartphones < 0) |>
  count(household_smartphones)
```

household_smartphones	n
-20	1
-19	1
-17	1
-16	2
-14	1
-12	1
-11	2
-5	1
-4	1
-3	1
-1	3

```
D08 |>
  left_join(household_size, by = "household_id") |>
  filter(household_smartphones >= 10) |>
  select(household_id, household_smartphones, household_size)
```

household_id	household_smartphones	household_size
167100600020000101	13	17
290105890000031501	10	13
526101040000015501	10	12
546100590000009301	10	12
546100590000020101	13	16
587100490000009101	10	14
599128110000011201	10	12
797101510000029701	10	13
798124950000021101	10	12
798125760000029501	12	25
798131470000016101	10	10

household_id	household_smartphones	household_size
798145240000023201	10	12

In the boxplot, the 15 points below the lower fence are exactly the negative counts. Most households have 1 or 2 smartphones, so the box is very narrow (an IQR of 1). Because of this, the $1.5 \times \text{IQR}$ rule also marks every household with 4 or more smartphones as an outlier, 2,275 households in total. These high points only stand out because the box is so narrow, not because they are errors, so they are compared with household size instead of being removed.

The negative values cannot be repaired. No other kept dataset records smartphones, and simply removing the minus sign is not justified, because 6 of them would then be above the dictionary maximum of 13. At the other end, every household with 10 or more smartphones has at least as many members as smartphones. `household_size` includes all members, including those outside the 15–64 range, because it was counted before record selection (section 5).

The negative counts are therefore set to NA, and `household_smartphones_flag` notes the original negative value on each of these rows. The high counts are realistic and stay as they are. The affected households are kept, since their other connectivity columns are still valid.

```
D08_negative_smartphones_before <- sum(D08$household_smartphones < 0, na.rm = TRUE)

D08 <- D08 |>
  mutate(
    household_smartphones_flag = if_else(
      household_smartphones < 0,
      paste0("invalid negative count (", household_smartphones, ") set to missing"),
      NA_character_,
    ),
    household_smartphones = if_else(
      household_smartphones < 0,
      NA_real_,
      household_smartphones
    )
  )
```

The check below compares the counts before and after the fix and redraws the boxplot:

```
D08 |>
  left_join(household_size, by = "household_id") |>
  summarise(
    negative_before = D08_negative_smartphones_before,
    negative_after = sum(household_smartphones < 0, na.rm = TRUE),
```

```

missing_after = sum(is.na(household_smartphones)),
rows_flagged = sum(!is.na(household_smartphones_flag)),
min_after = min(household_smartphones, na.rm = TRUE),
max_after = max(household_smartphones, na.rm = TRUE),
households_with_more_smartphones_than_members =
  sum(household_smartphones > household_size, na.rm = TRUE)
)

```

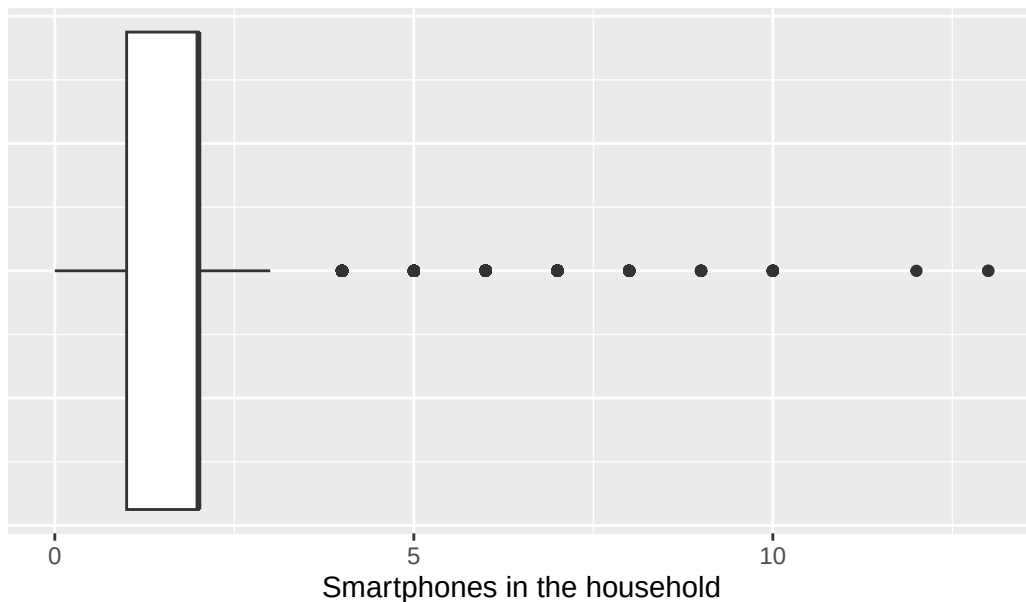
neg- ative_be- fore	nega- tive_af- ter	miss- ing_af- ter	rows_flagged	min_af- ter	max_af- ter	holds_with_more_smart- phones_than_members	house-
15	0	15	15	0	13		0

```

ggplot(D08, aes(x = household_smartphones)) +
  geom_boxplot() +
  labs(
    title = "Household smartphones after treatment (boxplot)",
    x = "Smartphones in the household",
    y = NULL) +
  theme(axis.text.y = element_blank(), axis.ticks.y = element_blank())

```

Household smartphones after treatment (boxplot)



The 15 impossible counts are now NA, their original values are kept as notes in `household_smartphones_flag`, and no household is removed. The boxplot no longer has any points below the box. All remaining counts fall within the dictionary range of 0–13, and no household has more smartphones than members, including the 2,275 households above the upper fence, which stay unchanged. `household_size` is only used for this check and is not joined onto D08 or saved.

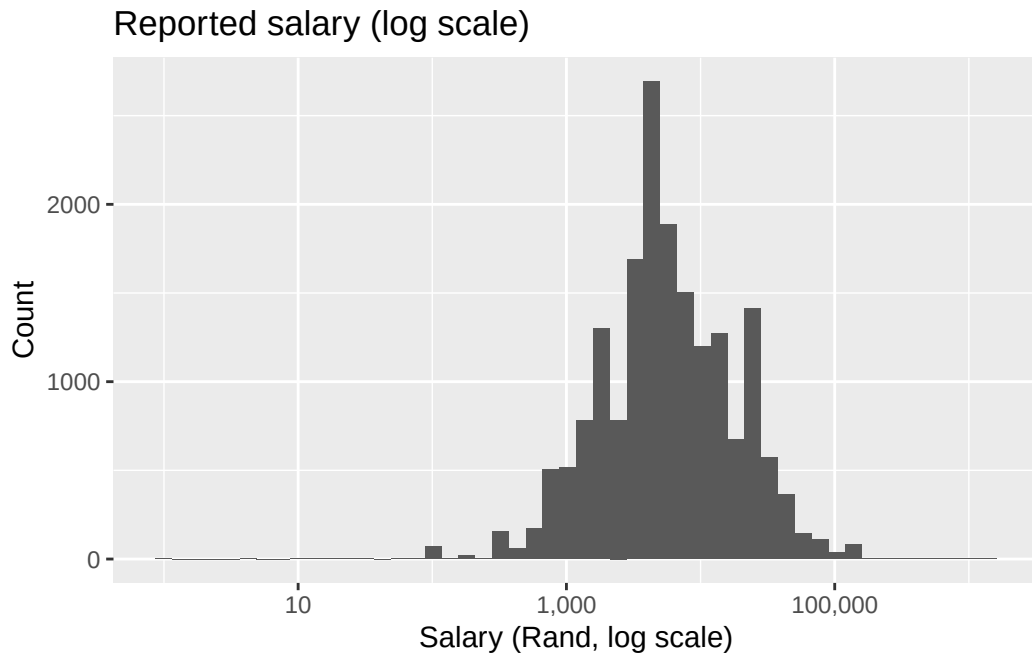
7.3 Salary

Reported salaries range from very small to very large amounts, and a few extreme values could pull averages up or down. Because the amounts are so spread out, the histogram and boxplot use a log scale:

```
D03 |> filter(!is.na(salary)) |>
  summarise(
    min = min(salary),
    p1 = quantile(salary, 0.01),
    median = median(salary),
    p99 = quantile(salary, 0.99),
    max = max(salary)
  )
```

min	p1	median	p99	max
1	360	5257	77319	1400000

```
D03 |>
  filter(!is.na(salary)) |>
  ggplot(aes(x = salary)) +
  geom_histogram(bins = 50) +
  scale_x_log10(labels = scales::comma) +
  labs(
    title = "Reported salary (log scale)",
    x = "Salary (Rand, log scale)",
    y = "Count"
  )
```

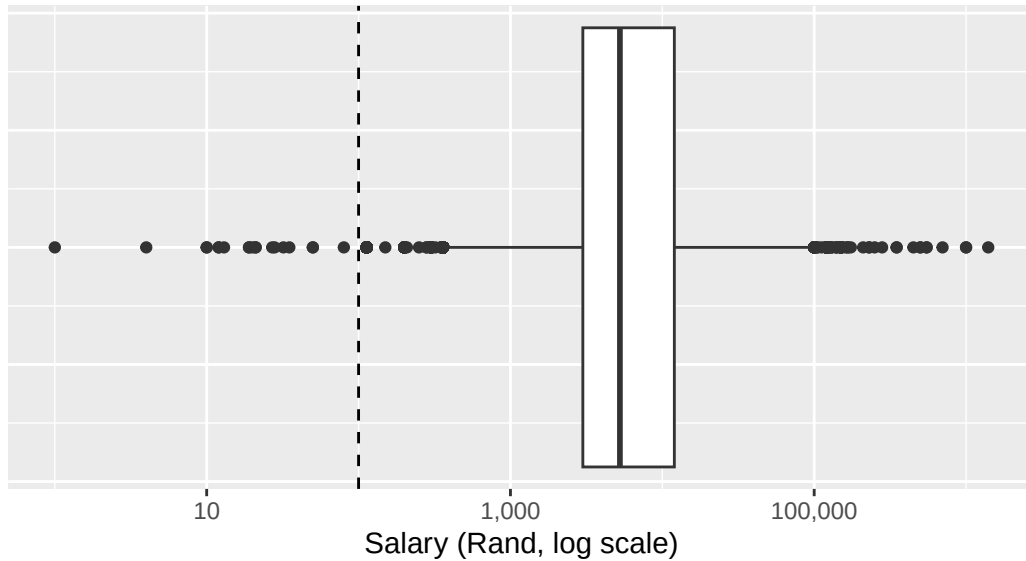


```
D03 |>
  filter(!is.na(salary)) |>

  ggplot(aes(x = salary)) +
    geom_boxplot() +
    geom_vline(xintercept = 100, linetype = "dashed") +
    scale_x_log10(labels = scales::comma) +
    labs(
      title = "Reported salary (boxplot, log scale)",
      subtitle = "Dashed line: R100 threshold used for salary_flag",
      x = "Salary (Rand, log scale)",
      y = NULL
    ) +
    theme(axis.text.y = element_blank(), axis.ticks.y = element_blank())
```

Reported salary (boxplot, log scale)

Dashed line: R100 threshold used for salary_flag



```
salary_boxplot <- D03 |>
  filter(!is.na(salary)) |>
  summarise(
    log_q1 = quantile(log10(salary), 0.25),
    log_q3 = quantile(log10(salary), 0.75),
    lower_fence_rand = 10^(log_q1 - 1.5 * (log_q3 - log_q1)),
    upper_fence_rand = 10^(log_q3 + 1.5 * (log_q3 - log_q1)),
    points_below_fence = sum(salary < lower_fence_rand),
    points_above_fence = sum(salary > upper_fence_rand),
    of_which_under_R100 = sum(salary < 100)
  ) |>
  select(-log_q1, -log_q3)
```

salary_boxplot

lower_fence_rand	upper_fence_rand	points_below_fence	points_above_fence	of_which_under_R100
374.0632	96144.18	266	133	19

On the log scale, the boxplot marks 266 salaries below about R374 and 133 above about R96,144 as outliers.

These outliers are not all handled the same way. The high ones, up to R1,400,000, are genuine high earners and are kept. The low ones are not all errors either: amounts between R100 and about R374 are low but possible for casual or part-time work, so they are kept without a note. Only the 19 values under R100 cannot realistically be a monthly salary. Rather than being deleted, they get the note “Implausibly low for a monthly salary” in `salary_flag`, because the source data does not document the pay period and the values might still be genuine. `salary_flag` uses a fixed R100 threshold rather than the boxplot fence, so it does not depend on the rest of the data.

```
D03 <- D03 |>
  mutate(
    salary_flag = if_else(
      !is.na(salary) & salary < 100,
      "Implausibly low for a monthly salary",
      NA_character_
    )
  )
```

The check below confirms that the note in `salary_flag` lines up exactly with the R100 threshold:

```
D03 |>
  filter(!is.na(salary)) |>
  summarise(
    reported_salaries = n(),
    flagged_under_R100 = sum(!is.na(salary_flag)),
    largest_flagged_salary = max(salary[!is.na(salary_flag)]),
    smallest_unflagged_salary = min(salary[is.na(salary_flag)])
  )
```

reported_salaries	flagged_un- der_R100	largest_flagged_salary	smallest_un- flagged_salary
18036	19	80	113

19 of 18,036 reported salaries have a note in `salary_flag`, and every salary value stays the same. Because the threshold is fixed, later analyses can include or exclude these rows with `salary < 100`, even without `salary_flag`.

`person_weight` is not touched, because survey weights should never be trimmed or scaled. No scaling or transformation is applied at this stage either, and the log scale above is only used for plotting. Transformations follow later, and anything that has to learn from the data waits until after the train/test split.

8. Validating and Saving the Cleaned Datasets

8.1 Checking the cleaned datasets

The cleaned datasets are used in every later task, so before saving them we check that the pipeline as a whole did what the earlier sections say it did. People should only have been removed by record selection, keys should still be unique, no duplicates should remain, and every remaining missing value should have a reason.

The first table compares each cleaned dataset with the summary recorded in 1.2, before any cleaning:

```
tibble(  
  
  dataset = c("D01", "D02", "D03", "D08"),  
  
  rows_uncleaned = uncleaned_summary$rows,  
  
  rows_cleaned = c(  
    nrow(D01),  
    nrow(D02),  
    nrow(D03),  
    nrow(D08)  
  ),  
  
  cols_uncleaned = uncleaned_summary$cols,  
  
  cols_cleaned = c(  
    ncol(D01),  
    ncol(D02),  
    ncol(D03),  
    ncol(D08)  
  ),  
  
  missing_values_uncleaned = uncleaned_summary$missing_values,  
  
  missing_values_cleaned = c(  
    sum(is.na(select(D01, -ends_with("_flag")))),  
    sum(is.na(select(D02, -ends_with("_flag")))),  
    sum(is.na(select(D03, -ends_with("_flag")))),  
    sum(is.na(select(D08, -ends_with("_flag"))))  
  ),  
)
```

```

distinct_keys_uncleaned = uncleaned_summary$distinct_keys,

distinct_keys_cleaned = c(
  nrow(distinct(D01, household_id, person_number)),
  nrow(distinct(D02, household_id, person_number)),
  nrow(distinct(D03, household_id, person_number)),
  n_distinct(D08$household_id)
),

exact_duplicates_uncleaned = uncleaned_summary$exact_duplicate_rows,

exact_duplicates_cleaned = c(
  nrow(D01) - nrow(distinct(D01)),
  nrow(D02) - nrow(distinct(D02)),
  nrow(D03) - nrow(distinct(D03)),
  nrow(D08) - nrow(distinct(D08))
)
)

```

									ex- act_du- pli- cates_un- cleaned	ex- act_du- pli- cates_cleaned
dataset	rows_un- cleaned	cols_un- cleaned	rows_cleaned	cols_cleaned	miss- ing_val- ues_un- cleaned	miss- ing_val- ues_cleaned	dis- tinct_keys_un- cleaned	dis- tinct_keys_cleaned		
D01	70440	45413	17	17	115	0	70422	45413	0	0
D02	70462	45413	23	12	120	578	70440	45413	22	0
D03	70460	45413	25	8	100	27377	70440	45413	20	0
D08	20940	20940	18	9	60	142	20940	20940	0	0

`missing_values_cleaned` leaves out the `_flag` columns, because they are empty on every row that was not changed and would make the missing counts look much higher than they are. D01 has no missing values left. The cleaned counts for D02, D03 and D08 are higher than the uncleaned ones, but this does not mean data was lost. Placeholders, non-response codes and impossible values were stored as ordinary values in the uncleaned data, so they were never counted as missing. They are now honest NAs:

- In D03, `salary` is NA for the placeholders replaced in 4.2, and nearly all of these belong to people who are not employed and cannot have a salary.
- In D02 and D08, the extra NAs come from the text placeholders set to missing in 2.3, the negative smartphone counts in 7.2, and the label columns from section 6, which carry over the missing values of their `_code` column and leave the non-response codes without a label.

The next table lists the columns that were added or changed type since column selection (section 3):

```
bind_rows(  
  D01 = enframe(  
    D01_types_before,  
    name = "column",  
    value = "type_before") |>  
  full_join(  
    enframe(  
      sapply(D01, class),  
      name = "column",  
      value = "type_after"  
    ),  
    by = "column"  
  ),  
  
  D02 = enframe(  
    D02_types_before,  
    name = "column",  
    value = "type_before"  
  ) |>  
  full_join(  
    enframe(  
      sapply(D02, class),  
      name = "column",  
      value = "type_after"),  
    by = "column"  
  ),  
  
  D03 = enframe(  
    D03_types_before,  
    name = "column",  
    value = "type_before"  
  ) |>  
  full_join(  
    enframe(  
      sapply(D03, class),  
      name = "column",  
      value = "type_after"  
    ),  
    by = "column"  
  ),  
)
```

```

D08 =
  enframe(
    D08_types_before,
    name = "column",
    value = "type_before"
  ) |>
  full_join(
    enframe(
      sapply(D08, class),
      name = "column",
      value = "type_after"
    ),
    by = "column"),
  .id = "dataset"
) |>
  filter(
    is.na(type_before) != is.na(type_after) |
      (!is.na(type_before) & !is.na(type_after) & type_before != type_after))

```

dataset	column	type_before	type_after
D01	province	NA	factor
D01	sex	NA	factor
D01	population_group	NA	factor
D01	geo_type	NA	factor
D01	metro	NA	factor
D02	education_level	NA	factor
D02	education_attendance	NA	factor
D02	institution_type	NA	factor
D02	reason_not_attending	NA	factor
D03	salary	NA	numeric
D03	salary_status	NA	character
D03	employment_status	NA	factor
D03	salary_flag	NA	character
D08	fixed_internet	NA	factor
D08	mobile_internet	NA	factor
D08	computer_asset	NA	factor
D08	household_smartphones_flag	NA	character

Every column that still has missing values, with the reason why:

```

final_missingness <- bind_rows(
  D01 = enframe(colSums(is.na(D01)), name = "column", value = "missing"),
  D02 = enframe(colSums(is.na(D02)), name = "column", value = "missing"),
  D03 = enframe(colSums(is.na(D03)), name = "column", value = "missing"),
  D08 = enframe(colSums(is.na(D08)), name = "column", value = "missing"),
  .id = "dataset"
) |>
filter(missing > 0) |>
mutate(
  reason = case_when(
    column == "education_level" ~ "Codes 29/99 reclassified to NA in sections 4/6",

    column == "education_attendance_code" ~ "Text placeholders ('N/A', 'Unknown', 'not rec

    column == "education_attendance" ~ "Code 3 reclassified to NA in sections 4/6, plus the

    column == "fixed_internet" ~ "Code 9 reclassified to NA in sections 4/6",

    column == "mobile_internet_code" ~ "Already missing in the uncleaned data; no dictionary

    column == "mobile_internet" ~ "Code 9 reclassified to NA in sections 4/6, plus the mis

    column == "computer_asset" ~ "Code 9 ('Unspecified') reclassified to NA in sections 4/6

    column == "household_smartphones" ~ "Negative counts set to NA and flagged in 7.2",

    str_detect(column, "_flag$") ~ "Flags only filled in for rows that changed (expected)"

    column == "salary" ~ "Placeholder codes reversed to NA in section 4",

    column == "salary_status" ~ "Could not be classified from value and employment status"

    TRUE ~ "NOT CLASSIFIED"
  )
)

final_missingness

```

dataset	column	missing	reason
D01	household_id_flag	45395	Flags only filled in for rows that changed (expected)
D01	age_flag	45340	Flags only filled in for rows that changed (expected)
D02	education_attendance_code	23	Text placeholders ('N/A', 'Unknown', 'not recorded') set to NA and flagged in 2.3
D02	education_level_flag	45333	Flags only filled in for rows that changed (expected)
D02	education_attendance_flag	45390	Flags only filled in for rows that changed (expected)
D02	education_level	514	Codes 29/99 reclassified to NA in sections 4/6
D02	education_attendance	41	Code 3 reclassified to NA in sections 4/6, plus the text placeholders set to NA in 2.3
D03	salary	27377	Placeholder codes reversed to NA in section 4
D03	salary_flag	45394	Flags only filled in for rows that changed (expected)
D08	mobile_internet_code	60	Already missing in the uncleaned data; no dictionary code to assign (noted in section 4)
D08	household_smartphones	15	Negative counts set to NA and flagged in 7.2
D08	fixed_internet	2	Code 9 reclassified to NA in sections 4/6
D08	mobile_internet	62	Code 9 reclassified to NA in sections 4/6, plus the missing mobile_internet_code values
D08	computer_asset	3	Code 9 ('Unspecified') reclassified to NA in sections 4/6
D08	household_smartphones_flag	20925	Flags only filled in for rows that changed (expected)

The levels of every factor column, in order. These are needed again whenever the saved CSVs are read back in, because a CSV does not store the level order:

```
#One row per factor column: which dataset it is in, how many levels it has, and the levels in order
bind_rows(
  D01 = enframe(
    lapply(select(D01, where(is.factor)), levels),
    name = "column",
    value = "levels"
  ),
  D02 = enframe(
    lapply(select(D02, where(is.factor)), levels),
    name = "column",
```

```

    value = "levels"
  ),

  D03 = enframe(
    lapply(select(D03, where(is.factor)), levels),
    name = "column",
    value = "levels"
  ),

  D08 = enframe(
    lapply(select(D08, where(is.factor)), levels),
    name = "column",
    value = "levels"),
  .id = "dataset"
) |>
mutate(
  number_of_levels = lengths(levels),
  levels = map_chr(levels, \(x) paste(x, collapse = " | "))
) |>
select(dataset, column, number_of_levels, levels)

```

	num-
	col- ber__of__lev-
	dataset levels
D01prov	9 Western Cape Eastern Cape Northern Cape Free State KwaZulu-Natal North West Gauteng Mpumalanga Limpopo
D01sex	2 Male Female
D01pop	4 Black African Coloured Indian/Asian White
D01u-	
D01la-	
D01tion	group
D01geo	3 Urban Traditional Farms
D01metr	2 Metro Non-metro

num- col-ber_of_lev- data_sets levels	
D02d-30	No schooling Grade R/0 Grade 1/Sub A/Class 1 Grade 2/Sub B/Class 2 Grade 3/Standard 1/ABET/AET 1 Grade 4/Standard 2 Grade 5/Standard 3/ABET/AET 2 Grade 6/Standard 4 Grade 7/Standard 5/ABET/AET 3 Grade 8/Standard 6/Form 1 Grade 9/Standard 7/Form 2/ABET/AET 4/NCV 1 Grade 10/Standard 8/Form 3/NCV 2 Grade 11/Standard 9/Form 4/NCV 3 Grade 12/Standard 10/Form 5/NSC/Matric/NCV 4 NTC I/N1/NQF 1 NTC II/N2/NQF 2 NTC III/N3/NQF 3 N4/NTC 4/Occupational Cert NQF 5 N5/NTC 5/Occupational Cert NQF 5 N6/NTC 6/Occupational Cert NQF 5 Certificate less than Grade 12 Diploma less than Grade 12 Higher/National/Advanced Certificate with Grade 12/NQF 5 Diploma with Grade 12/Occupational Cert NQF 6 Higher Diploma/B-Tech/Occupational Cert NQF 7 Bachelor's Degree/Occupational Cert NQF 7 Honours Degree/Postgrad Diploma/Occupational Cert NQF 8 Post Higher Diploma (M-Tech/Master's) NQF 9 Doctoral Degree (D-Tech/PhD) NQF 10 Other
D02d-2	Yes No
u- ca- tion_at- ten- dance	
D02n-10	Pre-school (incl. ECD centre) School (Grade R-12) ABET/AET centre Literacy classes Higher education institution (university / university of technology) TVET college Other college Home-based education / home schooling Other educational institution Not applicable
D02ea-19	Too old/young Completed education / satisfied / doesn't want to study Institution too far Difficulty getting there (transport) No money for fees Working (at home, business or job) Family commitment (e.g. child minding) Education useless / not interesting Unable to perform at school Illness Pregnancy Failed exams Got married Disability Violence in school Not accepted for enrolment No time / too busy Other reason Not applicable
D03m-3	Employed Unemployed Not economically active
ploy- ment_sta- tus	
D03fixed_2	Yes No
ter- net	

	num-
	col-ber_of_lev-
datasets	levels
D08mo-2	Yes No
	bile_in-
	ter-
	net
D08om-2	Yes No
	puter_as-
	set

D01 goes from 70,440 to 45,413 people. The difference is exactly the 25,027 people outside ages 15–64 removed in section 5, so nobody was lost during cleaning. D02 and D03 match D01 after also losing their 22 and 20 exact duplicate rows (2.2), and D08 keeps all 20,940 households. Every key appears exactly once in every dataset, and no exact duplicates remain. 0 of the 15 columns with missing values are without a documented reason.

8.2 Saving the cleaned datasets

The cleaned datasets are saved so that later tasks can use them without rerunning this whole pipeline. Each one is written as a new `.csv` file (a required deliverable) in `Datasets/Cleaned/`. The original files in `Datasets/` are only ever read, never written to.

A few details matter for anyone using these files:

- Missing values are written as empty cells rather than the text “NA”. Power BI then reads them as blanks, instead of turning numeric columns such as `salary` into text.
- A CSV does not store column types or the order of factor levels. Anyone reading the files, including Power BI, must read `household_id` and `person_number` as text, and re-apply the level order listed above where the order matters.
- The `_flag` columns are left out of the saved files, because they only keep track of the cleaning. They stay in the datasets while this pipeline runs, so the notes are still available for the checks in this report, and they are removed later, before modelling. The saved files only contain the analytical columns: the original codes, the cleaned values and the labels.

Saving at this point, before the train/test split in Task 4, does not leak information between the future training and test rows. Every change so far either applies a fixed rule to each row (dictionary codes, the 15–64 age range, the R100 salary threshold, negative counts) or copies a value from the same person’s record in another dataset. Nothing is calculated from the data as a whole: no means, quantiles, imputation models or scaling are used to change any value.

Steps that learn from the data belong after the split, where they are fitted on the training rows only.

```
#na = "" writes missing values as empty cells, which both R and Power BI read as missing
#select(-ends_with("_flag")) leaves the traceability flags out of the saved files; they stay
write_csv(
  select(D01, -ends_with("_flag")),
  file.path(cleaned_directory, "D01_clean.csv"),
  na = ""
)

write_csv(
  select(D02, -ends_with("_flag")),
  file.path(cleaned_directory, "D02_clean.csv"),
  na = ""
)

write_csv(
  select(D03, -ends_with("_flag")),
  file.path(cleaned_directory, "D03_clean.csv"),
  na = ""
)

write_csv(
  select(D08, -ends_with("_flag")),
  file.path(cleaned_directory, "D08_clean.csv"),
  na = ""
)
```

To make sure the files were written correctly, they are read back in with the IDs as text, the same way later tasks should read them. They are then compared with the datasets in memory (without their flag columns), and checked to confirm that no flag column was saved:

```
D01_saved <- read_csv(
  file.path(cleaned_directory, "D01_clean.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character(), person_number = col_character())
)

D02_saved <- read_csv(
  file.path(cleaned_directory, "D02_clean.csv"),
  show_col_types = FALSE,
```

```

col_types = cols(household_id = col_character(), person_number = col_character())
)

D03_saved <- read_csv(
  file.path(cleaned_directory, "D03_clean.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character(), person_number = col_character())
)

D08_saved <- read_csv(
  file.path(cleaned_directory, "D08_clean.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character())
)

tibble(

  dataset = c("D01", "D02", "D03", "D08"),

  rows_in_memory = c(
    nrow(D01),
    nrow(D02),
    nrow(D03),
    nrow(D08)
  ),

  rows_in_saved_file = c(
    nrow(D01_saved),
    nrow(D02_saved),
    nrow(D03_saved),
    nrow(D08_saved)
  ),

  flag_columns_in_saved_file = c(
    sum(str_ends(names(D01_saved), "_flag")),
    sum(str_ends(names(D02_saved), "_flag")),
    sum(str_ends(names(D03_saved), "_flag")),
    sum(str_ends(names(D08_saved), "_flag"))
  ),

  same_columns_apart_from_flags = c(
    identical(names(D01_saved), names(select(D01, -ends_with("_flag")))),

```

```

identical(names(D02_saved), names(select(D02, -ends_with("_flag")))),
identical(names(D03_saved), names(select(D03, -ends_with("_flag")))),
identical(names(D08_saved), names(select(D08, -ends_with("_flag"))))
),

ids_identical = c(
  identical(D01_saved$household_id, D01$household_id) &
    identical(D01_saved$person_number, D01$person_number),
  identical(D02_saved$household_id, D02$household_id) &
    identical(D02_saved$person_number, D02$person_number),
  identical(D03_saved$household_id, D03$household_id) &
    identical(D03_saved$person_number, D03$person_number),
  identical(D08_saved$household_id, D08$household_id)
),

missing_values_identical = c(
  sum(is.na(D01_saved)) == sum(is.na(select(D01, -ends_with("_flag")))),
  sum(is.na(D02_saved)) == sum(is.na(select(D02, -ends_with("_flag")))),
  sum(is.na(D03_saved)) == sum(is.na(select(D03, -ends_with("_flag")))),
  sum(is.na(D08_saved)) == sum(is.na(select(D08, -ends_with("_flag"))))
)
)

```

dataset	rows_in_memory	rows_in_saved_files	flag	columns_in_saved_files	columns_apart_from_flags	ids_identical	missing_values_identical
D01	45413	45413	0	TRUE	TRUE	TRUE	TRUE
D02	45413	45413	0	TRUE	TRUE	TRUE	TRUE
D03	45413	45413	0	TRUE	TRUE	TRUE	TRUE
D08	20940	20940	0	TRUE	TRUE	TRUE	TRUE

Four new files are written to `Datasets/Cleaned/`. These four `write_csv()` calls are the only place where the pipeline writes data, so the original files in `Datasets/` stay unchanged. None of the saved files contains a flag column, and apart from the flags, each file has the same rows, columns, IDs and missing values as the cleaned dataset in memory. Later tasks can therefore start from these files instead of rerunning the pipeline.

9. Data Integration

The analytical grain chosen in the Task 1 write-up is one row per working-age person (15–64), identified by `household_id + person_number`, and section 1.4 explains why only D01, D02, D03 and D08 are used. The datasets are only joined now, after cleaning, because the keys are only reliable once the missing IDs are recovered (2.1) and the duplicates removed (2.2), and the population is only final after record selection (section 5). D01 is the base table, because it now contains exactly this population. Before each join, this section confirms the key and the expected cardinality, and after each join it checks the row counts and any unmatched records.

9.1 Checking keys and cardinality before joining

A join can go wrong in two ways. If a key is not unique on the side that should only match once, rows get duplicated, and if keys are missing or do not match, people are silently dropped or attached to the wrong records. The keys of each dataset are therefore checked first:

```
tibble(  
  
  dataset = c("D01", "D02", "D03", "D08"),  
  
  key = c(  
    "household_id + person_number",  
    "household_id + person_number",  
    "household_id + person_number",  
    "household_id"  
  ),  
  
  rows = c(  
    nrow(D01),  
    nrow(D02),  
    nrow(D03),  
    nrow(D08)  
  ),  
  
  distinct_keys = c(  
    nrow(distinct(D01, household_id, person_number)),  
    nrow(distinct(D02, household_id, person_number)),  
    nrow(distinct(D03, household_id, person_number)),  
    n_distinct(D08$household_id)  
  ),  
)
```

```

missing_keys = c(
  sum(is.na(D01$household_id) | is.na(D01$person_number)),
  sum(is.na(D02$household_id) | is.na(D02$person_number)),
  sum(is.na(D03$household_id) | is.na(D03$person_number)),
  sum(is.na(D08$household_id))
),

all_ids_18_digits = c(
  all(nchar(D01$household_id) == 18),
  all(nchar(D02$household_id) == 18),
  all(nchar(D03$household_id) == 18),
  all(nchar(D08$household_id) == 18)
)
)

```

dataset	key	rows	dis- tinct_keys	miss- ing_keys	all_ids_18_dig- its
D01	household_id + person_number	45413	45413	0	TRUE
D02	household_id + person_number	45413	45413	0	TRUE
D03	household_id + person_number	45413	45413	0	TRUE
D08	household_id	20940	20940	0	TRUE

The next table shows the expected cardinality of each join, the most rows any key has on either side, and how many rows would go unmatched:

```

#Households in D08 with no working-age member, and whether they had members before record set
D08_households_without_D01 <- anti_join(D08, D01, by = "household_id")

tibble(

  join = c("D01 + D02", "D01 + D03", "D01 + D08"),

  expected_cardinality = c("one-to-one", "one-to-one", "many-to-one"),

  most_D01_rows_per_key = c(
    max(count(D01, household_id, person_number)$n),
    max(count(D01, household_id, person_number)$n),

```

```

    max(count(D01, household_id)$n)
  ),

  most_rows_per_key_in_joined_dataset = c(
    max(count(D02, household_id, person_number)$n),
    max(count(D03, household_id, person_number)$n),
    max(count(D08, household_id)$n)
  ),

  D01_rows_without_a_match = c(
    nrow(anti_join(D01, D02, by = c("household_id", "person_number"))),
    nrow(anti_join(D01, D03, by = c("household_id", "person_number"))),
    nrow(anti_join(D01, D08, by = "household_id"))
  ),

  joined_dataset_rows_without_a_match = c(
    nrow(anti_join(D02, D01, by = c("household_id", "person_number"))),
    nrow(anti_join(D03, D01, by = c("household_id", "person_number"))),
    nrow(D08_households_without_D01)
  )
)

```

join	ex- pected_car- dinality	most_D01_rows_per_key	most_rows_per_key_out_joined_dataset	D01_rows_without_a_match	joined_dataset_rows_without_a_match
D01 + D02	one-to-one	1	1	0	0
D01 + D03	one-to-one	1	1	0	0
D01 + D08	many-to-one	15	1	0	1160

```

tibble(
  D08_households_without_a_working_age_member = nrow(D08_households_without_D01),

  of_which_had_members_before_record_selection = sum(D08_households_without_D01$household_id
)

```

D08_households_without_a_working_age_member	of_which_had_members_before_record_selection
1160	1160

Every key is unique, complete and 18 digits long, and D01, D02 and D03 contain exactly the same people. Every person has a D08 household record, and each household appears only once in D08, so no person can be duplicated by the join. The only unmatched records are the 1,160 D08 households with no working-age member. All of them had members in D01 before record selection, which means all their members are under 15 or over 64. They fall outside the analysis population by design, not because of a key problem, and they remain available in `D08_clean.csv` for household-level description.

With this confirmed, the joins are set up as follows:

- D01 is the left table, so every working-age person is kept.
- D02 and D03 are joined one-to-one on `household_id + person_number`.
- D08 is joined many-to-one on `household_id`, so each person gets their own household's connectivity values.
- Each join states its expected cardinality in dplyr's `relationship` argument, which stops the pipeline with an error if any key matches more rows than expected.
- The D02 and D03 joins also use `unmatched = "error"`, which stops the pipeline if any of their rows would be dropped. D08 cannot use this option, because some of its households are expected to go unmatched (see above).

9.2 Joining the datasets

The three joins are run one at a time, so the row count can be checked before and after each one: D02 first, then D03, then D08, each onto D01.

```
integrated <- D01 |>
  left_join(
    D02,
    by = c("household_id", "person_number"),
    relationship = "one-to-one",
    unmatched = "error"
  )

tibble(
  join = "D01 + D02",
```

```

rows_before = nrow(D01),

rows_in_D02 = nrow(D02),

rows_after = nrow(integrated),

people_without_D02_match = sum(is.na(integrated$education_level_code)),

columns_after = ncol(integrated),

columns_expected = ncol(D01) + ncol(D02) - 2 #The two key columns appear once
)

```

join	rows_before	rows_in_D02	rows_after	people_without_D02_match	columns_after	columns_expected
D01 + D02	45413	45413	45413	0	27	27

```

integrated_rows_before_D03 <- nrow(integrated)

integrated <- integrated |>
  left_join(
    D03,
    by = c("household_id", "person_number"),
    relationship = "one-to-one",
    unmatched = "error"
  )

tibble(
  join = "+ D03",

  rows_before = integrated_rows_before_D03,

  rows_in_D03 = nrow(D03),

  rows_after = nrow(integrated),

  people_without_D03_match = sum(is.na(integrated$employment_status2code)),

  columns_after = ncol(integrated),

```

```
columns_expected = ncol(D01) + ncol(D02) - 2 + ncol(D03) - 2
)
```

	rows_be- fore	rows_in_D03	rows_af- ter	people_with- out_D03_match	columns_af- ter	columns_ex- pected
+ D03	45413	45413	45413	0	33	33

```
integrated_rows_before_D08 <- nrow(integrated)
```

```
integrated <- integrated |>
  left_join(
    D08,
    by = "household_id",
    relationship = "many-to-one"
  )
```

#computer_asset_code has no missing values in D08, so a missing value here would mean a person

```
tibble(
  join = "+ D08",

  rows_before = integrated_rows_before_D08,

  rows_in_D08 = nrow(D08),

  rows_after = nrow(integrated),

  people_without_D08_match = sum(is.na(integrated$computer_asset_code)),

  households_in_integrated_data = n_distinct(integrated$household_id),

  D08_households_not_joined = nrow(D08) - n_distinct(integrated$household_id),

  columns_after = ncol(integrated),

  columns_expected = ncol(D01) + ncol(D02) - 2 + ncol(D03) - 2 + ncol(D08) - 1
)
```

	rows_before	rows_in_D08	rows_after_D08	people_with_D08_matches	households_in_integrated_data	D08_households_not_joined	columns_added	columns_expected
+ D08	45413	20940	45413	0	19780	1160	41	41

The row count stays at 45,413 through all three joins, so no person is duplicated or lost. Every person matches a D02, D03 and D08 record, and no columns are duplicated. The only records not carried into the person-level dataset are the 1,160 households with no working-age member.

9.3 Checking the integrated dataset

Matching row counts do not prove that values ended up with the right people. A join on damaged keys (for example, IDs rounded to 15 digits as described in 1.1) could keep the row count the same while attaching one household's values to another.

The integrated dataset is therefore checked against its sources. Every person key should appear once, there should be no suffixed duplicate columns (`.x/.y`), and all members of a household should share a single D08 record:

```
tibble(
  rows = nrow(integrated),

  distinct_person_keys = nrow(distinct(integrated, household_id, person_number)),

  columns = ncol(integrated),

  suffixed_duplicate_columns = sum(str_detect(names(integrated), "\\..x$|\\.y$")),

  households = n_distinct(integrated$household_id),

  households_with_one_D08_record = nrow(
    distinct(
      integrated,
      household_id,
      fixed_internet_code,
      mobile_internet_code,
      computer_asset_code, household_smartphones
    )
  )
)
```

rows	distinct_per- son_keys	columns	suffixed_dupli- cate_columns	house- holds	holds_with_one_D08_record
45413	45413	41	0	19780	19780

The category counts should also match the source datasets exactly, counted per person for the D01, D02 and D03 columns and per household for the D08 columns:

```
D08_matched_households <- semi_join(D08, D01, by = "household_id")

tibble(
  column = c(
    "sex (D01, per person)",
    "age (D01, per person)",
    "education_level (D02, per person)",
    "education_attendance (D02, per person)",
    "employment_status (D03, per person)",
    "salary_status (D03, per person)",
    "fixed_internet (D08, per household)",
    "mobile_internet (D08, per household)",
    "computer_asset (D08, per household)",
    "household_smartphones (D08, per household)"
  ),

  identical_to_source = c(
    identical(count(integrated, sex), count(D01, sex)),
    identical(count(integrated, age), count(D01, age)),
    identical(count(integrated, education_level), count(D02, education_level)),
    identical(count(integrated, education_attendance),
      count(D02, education_attendance)),
    identical(count(integrated, employment_status), count(D03, employment_status)),
    identical(count(integrated, salary_status), count(D03, salary_status)),
    identical(count(distinct(integrated, household_id, fixed_internet),
      fixed_internet), count(D08_matched_households, fixed_internet)),
    identical(count(distinct(integrated, household_id, mobile_internet),
      mobile_internet), count(D08_matched_households, mobile_internet
    )),
    identical(count(distinct(integrated, household_id, computer_asset),
      computer_asset), count(D08_matched_households, computer_asset)),
    identical(count(distinct(integrated, household_id, household_smartphones),
      household_smartphones), count(D08_matched_households,
      household_smartphones))
  )
)
```

```
)
)
```

column	identical_to_source
sex (D01, per person)	TRUE
age (D01, per person)	TRUE
education_level (D02, per person)	TRUE
education_attendance (D02, per person)	TRUE
employment_status (D03, per person)	TRUE
salary_status (D03, per person)	TRUE
fixed_internet (D08, per household)	TRUE
mobile_internet (D08, per household)	TRUE
computer_asset (D08, per household)	TRUE
household_smartphones (D08, per household)	TRUE

The integrated dataset has 45,413 rows, one per working-age person, and 41 columns from D01, D02, D03 and D08. It covers 19,780 households, and every category count matches its source, so no value was duplicated or lost. Every join matched on complete, 18-digit keys, so each value belongs to the person or household it came from. Household-level values are repeated for each working-age member of the household, which is correct at the person grain. `person_weight` is the right survey weight at this grain; `household_weight` was dropped in section 3.

10. Feature Engineering, Transformation and Encoding

The Milestone 1 analytics goal is to model employment status from education level, digital access and the interaction between the two, while controlling for demographic and geographic factors, using a logistic regression and a tree-based method. This section turns the integrated dataset from section 9 into features for that goal:

- 10.1 removes the `_flag` columns.
- 10.2 recodes the yes/no columns to 0/1.
- 10.3 builds the engineered features.
- 10.4 applies the transformations and creates the modelling dataset.
- 10.5 encodes the modelling dataset for the two model types.
- 10.6 saves the analytical datasets.

Every rule in this section is fixed in advance: a code mapping, a constant or a threshold. Nothing is calculated from the data as a whole, so these steps can run before the train/test split in Task 4 without leaking information from the test rows into the training rows.

10.1 Removing the flag columns

The `_flag` columns hold the notes on which values were changed or marked in sections 2–7. They were needed for the checks in those sections, but they describe the cleaning rather than the people, so they are not predictors. A model could even end up learning which rows were repaired instead of a real relationship. Every column ending in `_flag` is therefore removed from the integrated dataset. The repairs themselves stay documented in sections 2–7.

```
flag_columns <- str_subset(names(integrated), "_flag$")

analytical <- integrated |> select(-ends_with("_flag"))

tibble(
  rows_before = nrow(integrated),
  rows_after = nrow(analytical),
  columns_before = ncol(integrated),
  columns_after = ncol(analytical),
  flag_columns_removed = paste(flag_columns, collapse = ", "),
  flag_columns_remaining = sum(str_ends(names(analytical), "_flag"))
)
```

rows_before	rows_after	columns_before	columns_after	flag_columns_removed	flag_columns_remaining
45413	45413	41	35	household_id_flag, age_flag, education_level_flag, education_attendance_flag, salary_flag, household_smartphones_flag	0

6 flag columns are removed and no rows are lost, so the analytical dataset starts with 35 columns.

10.2 Recoding yes/no columns to 0/1

The source codes yes/no questions as 1 = Yes and 2 = No, and the label factors from section 6 list “Yes” first. Used as they are, `as.numeric()` would give 1 and 2 instead of 1 and 0, and a model would use “Yes” as the baseline, which flips the direction of every odds ratio.

To fix this, a 0/1 integer column is created for each yes/no variable, where 1 = Yes and 0 = No, and for sex, 1 = Female. Code 9, which section 4 classified as non-response or undocumented, and any missing codes stay NA. The original `_code` and label columns are not changed.

```
#case_when() returns NA for any code not listed, so non-response codes stay missing
analytical <- analytical |>
  mutate(
    female = case_when(sex_code == 2 ~ 1L, sex_code == 1 ~ 0L),

    has_fixed_internet = case_when(
      fixed_internet_code == 1 ~ 1L,
      fixed_internet_code == 2 ~ 0L
    ),

    has_mobile_internet = case_when(
      mobile_internet_code == 1 ~ 1L,
      mobile_internet_code == 2 ~ 0L
    ),

    has_computer = case_when(
      computer_asset_code == 1 ~ 1L,
      computer_asset_code == 2 ~ 0L
    )
  )
```

Each code should map to exactly one 0/1 value:

```
analytical |> count(sex_code, sex, female)
```

sex_code	sex	female	n
1	Male	0	21952
2	Female	1	23461

```
analytical |> count(fixed_internet_code, fixed_internet, has_fixed_internet)
```

fixed_internet_code	fixed_internet	has_fixed_internet	n
1	Yes	1	6687
2	No	0	38723
9	NA	NA	3

fixed_internet_code	fixed_internet	has_fixed_internet	n
---------------------	----------------	--------------------	---

```
analytical |> count(mobile_internet_code, mobile_internet, has_mobile_internet)
```

mobile_internet_code	mobile_internet	has_mobile_internet	n
1	Yes	1	35996
2	No	0	9275
9	NA	NA	3
NA	NA	NA	139

```
analytical |> count(computer_asset_code, computer_asset, has_computer)
```

computer_asset_code	computer_asset	has_computer	n
1	Yes	1	10535
2	No	0	34873
9	NA	NA	5

Four 0/1 columns are added. 1 always means “Yes” (or Female), so model coefficients read in the natural direction.

10.3 Engineered features

Each feature below is supported by the employment pattern it captures. To show this pattern, a helper function summarises employment for every level of a feature. It uses `person_weight`, so each person counts for the number of people they represent in the population:

- `percent_employed`: the share employed among all people aged 15–64.
- `employment_rate_in_labour_force`: the share employed among people who are employed or unemployed, which is the modelling target in 10.4.

```
#Weighted employment percentages for each level of a feature (used only to justify features)
employment_share <- function(data, feature) {
  data |>
    group_by({{ feature }}) |>
    summarise(
      people = n(),
```

```

percent_employed = round(sum(person_weight[employment_status == "Employed"]) / sum(per

employment_rate_in_labour_force = round(
  sum(person_weight[employment_status == "Employed"]) / sum(person_weight[employment_st
  .groups = "drop"
)
}

```

10.3.1 Education: education_band and matric_plus

education_level has 31 levels, many with very few people, and its level order follows the dictionary rather than a ranking (section 6). A model cannot estimate reliable effects for such small levels, and the hypotheses are about levels of education, not individual certificates. Two simpler features are built instead:

- **education_band** groups explicit codes into six ordered stages of education, plus an “Other/unknown” group. Codes 13–15 (NTC I–III) and 19–20 (certificate or diploma without Grade 12) are below matric, so they go into “Some secondary”. Code 28 (“Other”) is too small to stand on its own and cannot be ranked, so it joins the non-response and undocumented codes from section 4 (29 and 99) in “Other/unknown”.
- **matric_plus** is 1 for matric or any post-school qualification, and 0 for anything below matric. It is NA for “Other/unknown”, because that education cannot be placed above or below matric.

#Codes that are not listed would become NA, and the verification below confirms there are no

```

analytical <- analytical |>
mutate(
  education_band = case_when(
    education_level_code == 98 ~ "No schooling",
    education_level_code %in% 0:7 ~ "Primary (Grade R-7)",
    education_level_code %in% c(8:11, 13:15, 19:20) ~ "Some secondary",
    education_level_code == 12 ~ "Matric",
    education_level_code %in% c(16:18, 21:22) ~ "Post-school certificate/diploma",
    education_level_code %in% 23:27 ~ "Degree",
    education_level_code %in% c(28, 29, 99) ~ "Other/unknown"
  ),

  education_band = factor(
    education_band,
    levels = c(
      "No schooling",

```

```

    "Primary (Grade R-7)",
    "Some secondary",
    "Matric",
    "Post-school certificate/diploma",
    "Degree", "Other/unknown")
  ),

  matric_plus = case_when(
    education_level_code %in% c(12, 16:18, 21:27) ~ 1L,
    education_level_code %in% c(98, 0:11, 13:15, 19:20) ~ 0L
  )
)

```

Every education code should belong to exactly one band, and `matric_plus` should agree with the bands:

```

analytical |>
  group_by(education_band, matric_plus) |>
  summarise(education_level_codes = paste(
    sort(unique(education_level_code)), collapse = ", "),
    people = n(), .groups = "drop")

```

education_band	matric_plus	education_level_codes	people
No schooling	0	98	782
Primary (Grade R-7)	0	0, 1, 2, 3, 4, 5, 6, 7	4504
Some secondary	0	8, 9, 10, 11, 13, 14, 15, 19, 20	19239
Matric	1	12	14633
Post-school certificate/diploma	1	16, 17, 18, 21, 22	2958
Degree	1	23, 24, 25, 26, 27	2537
Other/unknown	NA	28, 29, 99	760

```

tibble(
  people_without_a_band = sum(is.na(analytical$education_band))
)

```

people_without_a_band
0

```
employment_share(analytical, education_band)
```

education_band	people	percent_employed	employment_rate_in_labour_force
No schooling	782	28.2	73.7
Primary (Grade R-7)	4504	32.2	64.1
Some secondary	19239	31.8	58.0
Matric	14633	49.5	64.5
Post-school certificate/diploma	2958	67.9	78.5
Degree	2537	82.1	89.7
Other/unknown	760	55.4	80.6

The 31 sparse education levels become 7 bands, and nobody is left without a band. Below matric, the three bands have similar employment percentages, but from matric upward the percentage rises sharply with every band. This jump at matric is why **matric_plus** is used as the single education term in the interaction with digital access.

In the labour force, the lowest bands show higher employment rates than “Some secondary”. These bands are mostly older people, and few of them are in the labour force, so this reflects age rather than education. It is one of the reasons age is controlled for in the model (10.4, T2).

10.3.2 Digital access: any_home_internet and digital_access_score

Digital access is spread over four columns. Mobile internet is very common, so on its own it does not separate people well, while fixed internet and computers are rare. A single yes/no column hides how much access a household has, but four separate columns make the interaction test hard to read. Two combined features are built:

- **any_home_internet** is 1 if the household has fixed or mobile internet, and 0 only if both are “No”. It is NA if one is “No” and the other is missing.
- **digital_access_score** counts how many forms of access the household has out of four: fixed internet, mobile internet, a computer and at least one smartphone (0–4). It is NA if any of the four is missing.

```
analytical <- analytical |>
  mutate(
    any_home_internet = case_when(
      has_fixed_internet == 1 | has_mobile_internet == 1 ~ 1L,
      has_fixed_internet == 0 & has_mobile_internet == 0 ~ 0L
    )
  )
```

```

),

digital_access_score = has_fixed_internet + has_mobile_internet + has_computer + as.integer(
)

```

The counts below check how the internet columns combine, and show employment for each level of the new features:

```
analytical |> count(has_fixed_internet, has_mobile_internet, any_home_internet)
```

has_fixed_internet	has_mobile_internet	any_home_internet	n
0	0	0	7725
0	1	1	30872
0	NA	NA	126
1	0	1	1550
1	1	1	5124
1	NA	1	13
NA	NA	NA	3

```
analytical |> count(digital_access_score)
```

digital_access_score	n
0	3765
1	4269
2	25963
3	7385
4	3851
NA	180

```
employment_share(analytical, any_home_internet)
```

any_home_internet	people	percent_employed	employment_rate_in_labour_force
0	7725	34.3	58.5
1	37559	46.0	67.6
NA	129	40.4	68.8

```
employment_share(analytical, digital_access_score)
```

digital_access_score	people	percent_employed	employment_rate_in_labour_force
0	3765	31.9	55.5
1	4269	35.5	59.9
2	25963	39.4	59.7
3	7385	54.0	76.6
4	3851	66.6	88.0
NA	180	42.9	68.8

`any_home_internet` is 1 only when at least one internet column is “Yes”, and 0 only when both are “No”. `digital_access_score` is missing for 180 people whose household has a missing digital-access answer. The weighted employment percentages rise with the score, with the biggest increases at scores 3 and 4, so the score picks up a gradient that a single yes/no column would miss.

10.3.3 Education × digital access: `edu_digital_group`

The central question is whether internet access strengthens the effect of education on employment (reinforcement) or takes its place (substitution). A logistic regression tests this with the interaction term `matric_plus * any_home_internet`, but tree models and Power BI need the combination as a single column.

`matric_plus` and `any_home_internet` are therefore combined into four groups. People whose education is “Other/unknown” get their own level, so they are not silently dropped. The group is only NA when internet access is unknown.

```
analytical <- analytical |>
  mutate(
    edu_digital_group = case_when(
      is.na(any_home_internet) ~ NA_character_,
      is.na(matric_plus) ~ "Education unknown",
      matric_plus == 0 & any_home_internet == 0 ~ "Below matric, no internet",
      matric_plus == 0 & any_home_internet == 1 ~ "Below matric, internet",
      matric_plus == 1 & any_home_internet == 0 ~ "Matric+, no internet",
      matric_plus == 1 & any_home_internet == 1 ~ "Matric+, internet"
    ),

    edu_digital_group = factor(
      edu_digital_group,
```

```

levels = c(
  "Below matric,
  no internet",
  "Below matric, internet",
  "Matric+, no internet",
  "Matric+, internet", "Education unknown")
)
)

```

The first table checks the mapping into groups. The second shows employment for each group, and the third calculates the internet gap in the labour-force employment rate within each education group:

```

analytical |> count(matric_plus, any_home_internet, edu_digital_group)

```

matric_plus	any_home_internet	edu_digital_group	n
0	0	NA	5692
0	1	Below matric, internet	18744
0	NA	NA	89
1	0	Matric+, no internet	1871
1	1	Matric+, internet	18218
1	NA	NA	39
NA	0	Education unknown	162
NA	1	Education unknown	597
NA	NA	NA	1

```

edu_digital_rates <- employment_share(analytical, edu_digital_group)
edu_digital_rates

```

edu_digital_group	people	percent_employed	employment_rate_in_labour_force
Below matric, internet	18744	32.0	59.9
Matric+, no internet	1871	42.0	59.7
Matric+, internet	18218	58.3	71.7
Education unknown	759	55.4	80.5
NA	5821	31.1	57.4

```
internet_gap <-
  tibble(

    internet_gap_below_matric =
      edu_digital_rates$employment_rate_in_labour_force[edu_digital_rates$edu_digital_group %in% "Below matric"] -
      edu_digital_rates$employment_rate_in_labour_force[edu_digital_rates$edu_digital_group %in% "At or above matric"]

    internet_gap_matric_plus =
      edu_digital_rates$employment_rate_in_labour_force[edu_digital_rates$edu_digital_group %in% "At or above matric"] -
      edu_digital_rates$employment_rate_in_labour_force[edu_digital_rates$edu_digital_group %in% "Below matric"]
  )
internet_gap
```

internet_gap_below_matric	internet_gap_matric_plus
---------------------------	--------------------------

Each combination of `matric_plus` and `any_home_internet` maps to exactly one group. In the labour force, people with internet have an employment rate percentage points higher than those without it below matric, and points higher at matric or above. Internet is linked to a bigger employment advantage for people with matric, which is the reinforcing pattern described in Milestone 1. The interaction term tests whether this difference between the two gaps is real. This section only builds the feature; whether the difference is statistically significant is tested in Milestone 3.

10.3.4 Location and age group: `settlement_type` and youth

`geo_type` and `metro` both describe where a person lives, spread over two overlapping columns, and some of their combinations are very small (metro farms and metro traditional areas). The policy discussion also refers to youth, which no existing column identifies. Two features handle this:

- `settlement_type` combines the two columns into four levels. Traditional areas and farms keep their own level whether or not they fall inside a metro, and urban areas are split into metro and non-metro.
- `youth` is 1 for ages 15–34, the South African definition of youth, and is used for reporting in Power BI.

```
analytical <- analytical |>
  mutate(
    settlement_type = case_when(
      geo_type_code == 2 ~ "Traditional",
```

```

    geo_type_code == 3 ~ "Farms",
    geo_type_code == 1 & metro_indicator == 1 ~ "Metro urban",
    geo_type_code == 1 & metro_indicator == 2 ~ "Non-metro urban"
  ),

  settlement_type = factor(
    settlement_type,
    levels = c(
      "Metro urban",
      "Non-metro urban",
      "Traditional",
      "Farms")
  ),

  youth = as.integer(age <= 34)
)

```

The tables below check both mappings and show employment by settlement type:

```

analytical |>
  count(geo_type, metro, settlement_type)

```

geo_type	metro	settlement_type	n
Urban	Metro	Metro urban	16962
Urban	Non-metro	Non-metro urban	12368
Traditional	Metro	Traditional	862
Traditional	Non-metro	Traditional	13690
Farms	Metro	Farms	127
Farms	Non-metro	Farms	1404

```

analytical |>
  group_by(youth) |>
  summarise(min_age = min(age), max_age = max(age), people = n())

```

youth	min_age	max_age	people
0	35	64	22616
1	15	34	22797

```
employment_share(analytical, settlement_type)
```

settlement_type	people	percent_employed	employment_rate_in_labour_force
Metro urban	16962	52.3	71.5
Non-metro urban	12368	46.6	69.4
Traditional	14552	28.6	51.7
Farms	1531	52.7	72.0

Every combination of `geo_type` and `metro` maps to one settlement type, with no missing values. Employment percentages differ clearly between settlement types, so the combined column keeps the location signal in a single variable.

10.3.5 Household features

Employment also depends on the household a person lives in. Caring for children or older family members can keep people, especially women, out of work, and living with educated household members may help someone find work. Smartphone counts also mean little without household size: three phones for one person is not the same as three phones for eight people. None of this is measured directly in the data.

To capture it, `household_composition` (counted in section 5, before the age filter) is joined many-to-one on `household_id`, and the following features are built:

- `n_working_age`: the number of people aged 15–64 in the household.
- `elderly_in_household`: 1 if anyone aged 65+ lives in the household.
- `dependency_ratio`: $(\text{children} + \text{people aged 65+}) \div \text{working-age members}$.
- `female_x_children`: $\text{female} \times \text{n_children_under15}$, so the effect of children can differ between women and men.
- `smartphones_per_member`: $\text{smartphones} \div \text{all household members}$.
- `smartphone_access`: the ratio grouped as “None”, “Fewer than 1 per member” or “1 or more per member”.
- `other_member_matric`: whether another working-age member of the household has matric or higher. Members with unknown education count as not having matric.

```
analytical_rows_before_household_join <- nrow(analytical)

analytical <- analytical |>
  left_join(
    household_composition,
    by = "household_id",
```

```

    relationship = "many-to-one") |>
add_count(household_id, name = "n_working_age") |>
group_by(household_id) |>
mutate(matric_members_in_household = sum(matric_plus == 1, na.rm = TRUE)) |>
ungroup() |>
mutate(
  elderly_in_household = as.integer(n_elderly_65plus > 0),

  dependency_ratio = (n_children_under15 + n_elderly_65plus) / n_working_age,

  female_x_children = female * n_children_under15,

  smartphones_per_member = household_smartphones / household_size,

  smartphone_access = case_when(
    household_smartphones == 0 ~ "None",
    smartphones_per_member < 1 ~ "Fewer than 1 per member",
    smartphones_per_member >= 1 ~ "1 or more per member"
  ),

  smartphone_access = factor(smartphone_access, levels = c("None", "Fewer than 1 per member", "1 or more per member")),

  other_member_matric = case_when(
    n_working_age == 1 ~ "No other working-age member",
    matric_members_in_household - coalesce(matric_plus, 0L) > 0 ~ "Yes",
    .default = "No"
  ),

  other_member_matric = factor(
    other_member_matric,
    levels = c("No", "Yes", "No other working-age member"))
) |>
select(-matric_members_in_household)

```

The join should not add or lose any people, and every household size should equal its children plus working-age members plus people aged 65+. The tables after that show employment for each household feature, including the number of children split by sex for people aged 25–54:

```

tibble(
  rows_before_join = analytical_rows_before_household_join,

  rows_after_join = nrow(analytical),

```

```

people_without_household_composition = sum(is.na(analytical$household_size)),

household_size_equals_sum_of_age_groups = all(
  analytical$household_size == analytical$n_children_under15 + analytical$n_working_age +

smallest_dependency_ratio = min(analytical$dependency_ratio),

largest_dependency_ratio = max(analytical$dependency_ratio),

highest_smartphones_per_member = max(analytical$smartphones_per_member, na.rm = TRUE)
)

```

rows_before_join	rows_after_join	people_without_household_composition	household_size_equals_sum_of_age_groups	smallest_dependency_ratio	largest_dependency_ratio	highest_smartphones_per_member
45413	45413	0	TRUE	0	8	1

```
analytical |> count(smartphone_access)
```

smartphone_access	n
None	4626
Fewer than 1 per member	28721
1 or more per member	12030
NA	36

```
employment_share(mutate(analytical, children = pmin(n_children_under15, 3)), children)
```

children	people	percent_employed	employment_rate_in_labour_force
0	18315	53.8	74.2
1	10827	42.8	65.4
2	8108	41.3	63.7
3	8163	28.9	50.1

```

analytical |>
  filter(between(age, 25, 54)) |>

```

```
mutate(children = pmin(n_children_under15, 3)) |>
group_by(sex) |>
group_modify(~ employment_share(.x, children)) |>
ungroup()
```

sex	children	people	percent_employed	employment_rate_in_labour_force
Male	0	7143	67.0	76.2
Male	1	2639	64.6	74.6
Male	2	1944	64.2	74.9
Male	3	1583	50.4	62.4
Female	0	4266	62.7	75.8
Female	1	3808	52.4	65.8
Female	2	3111	48.5	63.5
Female	3	3197	34.3	48.3

```
employment_share(analytical, elderly_in_household)
```

elderly_in_household	people	percent_employed	employment_rate_in_labour_force
0	37642	46.8	68.4
1	7771	30.0	52.5

```
employment_share(analytical, smartphone_access)
```

smartphone_access	people	percent_employed	employment_rate_in_labour_force
None	4626	33.3	57.0
Fewer than 1 per member	28721	37.7	59.9
1 or more per member	12030	62.8	80.6
NA	36	45.8	62.8

```
employment_share(analytical, other_member_matric)
```

other_member_matric	people	percent_employed	employment_rate_in_labour_force
No	14148	37.2	58.9

other_member_matric	people	percent_employed	employment_rate_in_labour_force
Yes	24650	44.1	67.0
No other working-age member	6615	62.2	78.1

The row count stays at 45,413, every person receives their household’s composition, and every household size equals the sum of its age groups. The tables show the patterns these features capture: employment falls as the number of children rises, and more steeply for women than for men. Smartphone access and having an educated household member are also associated with employment. `smartphone_access` is only missing for households whose smartphone count was set to missing in section 7.2.

10.4 Transformations

T1 Target population: the labour force

`employment_status` has three levels. The “Not economically active” group is mostly students and people close to retirement who are not looking for work, so the digital job-search mechanism in the business problem does not apply to them. A binary target also gives the odds ratios required by the technical success criteria.

The modelling dataset therefore keeps only the labour force: people who are employed or unemployed. The target `employed` is created when the modelling dataset is built below, and encoded in 10.5. The full working-age population stays in `analytical` for description and Power BI.

T2 Centred age and age squared

Employment does not change with age in a straight line, but a single age term can only fit a straight line. Adding `age`² allows a curve, but `age` and `age`² are almost perfectly correlated, which makes their regression coefficients unstable.

To solve both problems, a fixed 40 is subtracted from age (close to the middle of the 15–64 range) to create `age_centred`, which is then squared to create `age_centred_squared`. A fixed constant is used instead of the sample mean, so nothing is calculated from the data and this step cannot leak test information into training. Tree models can use `age_centred` directly, because centring does not change the order of the ages.

```
analytical <- analytical |>
  mutate(
    age_centred = age - 40,

    age_centred_squared = age_centred^2
  )
```

The check compares the correlation before and after centring, and shows employment across age bands:

```
tibble(
  correlation_age_with_age_squared = round(cor(analytical$age, analytical$age^2), 3),

  correlation_centred_age_with_its_square = round(cor(analytical$age_centred, analytical$age_centred^2), 3),

  min_age_centred = min(analytical$age_centred),

  max_age_centred = max(analytical$age_centred)
)
```

correlation_age_with_age_squared	correlation_centred_age_with_its_square	min_age_centred	max_age_centred
0.985	-0.327	-25	24

```
analytical |>
  mutate(
    age_band = cut(age, breaks = c(14, 24, 34, 44, 54, 64), labels = c("15-24", "25-34", "35-44", "45-54", "55-64"))
  ) |>
  employment_share(age_band)
```

age_band	people	percent_employed	employment_rate_in_labour_force
15-24	12140	11.6	34.8
25-34	10657	49.5	61.1
35-44	9705	61.0	72.8
45-54	7329	64.2	79.2
55-64	5582	41.3	84.1

Centring reduces the correlation between the age terms from 0.985 to -0.327. The age bands show why a curve is needed. Among all 15–64-year-olds, employment rises until 45–54 and

then falls. In the labour force, the rate jumps from 15–24 to 25–34 and then rises more slowly. Neither pattern is a straight line, and the squared term lets a logistic regression follow the curve.

T6 Log salary (income question only)

Reported salaries are strongly right-skewed (section 7.3), so a few very high salaries would dominate an income model. The business success criteria ask whether internet access is linked to higher income among people who report a salary, so the income model needs a less skewed version of salary.

`log_salary` is the natural log of `salary` for employed people with a reported salary of at least R100, since section 7.3 found that amounts under R100 cannot realistically be a monthly salary. Everyone else gets NA, and `salary` itself is not changed.

```
analytical <- analytical |>
  mutate(
    log_salary = if_else(
      employment_status == "Employed" &
        salary_status == "Reported" &
        salary >= 100, log(salary),
      NA_real_
    )
  )
```

The summary compares the mean and median on both scales, and the histogram shows the shape of the log salaries:

```
analytical |>
  filter(!is.na(log_salary)) |>
  summarise(
    people = n(),

    salary_mean = round(mean(salary)),

    salary_median = median(salary),

    salary_mean_divided_by_median = round(mean(salary) / median(salary), 2),

    log_salary_mean = round(mean(log_salary), 2),

    log_salary_median = round(median(log_salary), 2)
  )
```

people	salary_mean	salary_median	salary_mean_divided_by_median	log_salary_mean	log_salary_median
18017	10767	5257	2.05	8.62	8.57

```
analytical |>
  filter(!is.na(log_salary)) |>
  ggplot(aes(x = log_salary)) +
  geom_histogram(bins = 50) +
  labs(title = "Log salary (employed people with a reported salary of R100 or more)", x = "log(salary)")
```



On the Rand scale the mean is about twice the median, a clear sign of skew. On the log scale the mean and median are almost the same and the histogram is roughly bell-shaped, so no small group of high earners dominates the income model. The spikes in the histogram are most likely round amounts that people report (for example R5,000 or R10,000). They are genuine answers and are left as they are.

T3, T4, T5 and leakage: building the modelling dataset

The modelling dataset should only contain the target population (T1), usable predictors and nothing that leaks the outcome. Two columns give the outcome away directly:

- `reason_not_attending` = “Working” can only be the answer of someone who works.

- The salary columns only exist for employed people.

The tables below show how closely these columns line up with employment status:

```
analytical |>
  filter(employment_status != "Not economically active") |>
  mutate(reason_not_attending_is_working = reason_not_attending == "Working (at home, business, or other)") |>
  count(reason_not_attending_is_working, employment_status) |>
  pivot_wider(names_from = employment_status, values_from = n, values_fill = 0)
```

reason_not_attending_is_working	Employed	Unemployed
FALSE	9759	9939
TRUE	8952	196

```
analytical |>
  filter(employment_status != "Not economically active") |>
  count(salary_status, employment_status) |>
  pivot_wider(names_from = employment_status, values_from = n, values_fill = 0)
```

salary_status	Employed	Unemployed
Non-response (employed)	675	0
Not applicable (not employed)	0	10135
Reported	18036	0

The modelling dataset is then built with the following decisions:

- **T1:** only the labour force is kept.
- **T3:** unknown education is already kept as the “Other/unknown” level of `education_band` (10.3.1). These people are not missing at random, so dropping them would bias the sample.
- **T4:** the few people whose household has a missing digital-access answer are dropped. There is no dictionary code to give them, and they are too few to justify an imputation model.
- **T5:** rare levels were already merged by fixed rules in 10.3: “Other” education into “Other/unknown”, and metro farms and traditional areas into “Farms” and “Traditional”. No level is merged based on counts in the data.
- **Leakage:** only the target, the predictors and two design columns are kept. `household_id` is needed for the household-grouped train/test split in Task 4, and `person_weight` is the survey weight. Neither is used as a predictor.

```
labour_force_rows <- sum(analytical$employment_status != "Not economically active")

model_data <- analytical |>
  filter(employment_status %in% c("Employed", "Unemployed")) |>
  filter(!is.na(any_home_internet), !is.na(digital_access_score)) |>
  transmute(
    #Design columns: household-grouped split and survey weights in Task 4, never predictors
    household_id, person_weight,

    #Target
    employed = employment_status,

    #Demographic and location predictors
    age_centred, age_centred_squared, female, population_group, province, settlement_type,

    #Education predictors
    education_band, matric_plus,

    #Digital access predictors
    has_fixed_internet, has_mobile_internet, has_computer, any_home_internet, digital_access_score,

    #Education x digital access
    edu_digital_group,

    #Household predictors
    household_size, n_children_under15, elderly_in_household, dependency_ratio, female_x_children
  )
```

The checks below show the row counts at each step, the missing values that remain, and the reason every other column was left out:

```
tibble(
  working_age_people = nrow(analytical),

  labour_force = labour_force_rows,

  dropped_for_missing_digital_access = labour_force_rows - nrow(model_data),

  rows_in_model_data = nrow(model_data),

  columns_in_model_data = ncol(model_data),
```

```

missing_values_excluding_matric_plus = sum(is.na(select(model_data, -matric_plus))),

missing_matric_plus = sum(is.na(model_data$matric_plus)),

employed = sum(model_data$employed == "Employed"),

unemployed = sum(model_data$employed == "Unemployed"),

percent_employed = round(mean(model_data$employed == "Employed") * 100, 1)
)

```

work- ing_age_peo- ple	dropped_for_miss- ing_digi- labour_force_access	miss- ing_val- ues_exclud- ing_matric_plus	miss- ing_matric_plus	un- em- ployed	per- cent_em- ployed			
45413	28846	104	28742	24	2888	497	1864110101	64.9

```

excluded_columns <- tibble(column = setdiff(names(analytical), names(model_data))) |>
mutate(
  reason = case_when(
    column %in% c("reason_not_attending", "reason_not_attending_code", "lab_salary_raw", "
    column == "employment_status" ~ "Target: kept as employed",
    column %in% c("person_number", "psu") ~ "Identifier or survey design column",
    str_ends(column, "_code") | column %in% c("employment_status2code", "metro_indicator")
    column == "education_level" ~ "Replaced by education_band (31 sparse levels)",
    column == "sex" ~ "Replaced by female (0/1)",
    column %in% c("geo_type", "metro") ~ "Replaced by settlement_type",
    column == "age" ~ "Replaced by age_centred and age_centred_squared",
    column == "youth" ~ "Reporting group for Power BI; duplicates age in a model",
    column %in% c("fixed_internet", "mobile_internet", "computer_asset") ~ "Replaced by its",
    column %in% c("household_smartphones", "smartphones_per_member") ~ "Replaced by smartph

```

```

column %in% c("n_elderly_65plus", "n_working_age") ~ "Used to build elderly_in_household"

column %in% c("education_attendance", "institution_type") ~ "Current study status: mostly outside the labour force, kept for description"

TRUE ~ "NOT CLASSIFIED" #Should never appear; shows up if an excluded column has no reason
)
)

excluded_columns |>
  group_by(reason) |>
  summarise(columns = paste(column, collapse = ", "), .groups = "drop")

```

reason	columns
Code column: its label or an engineered feature is used instead	province_code, sex_code, population_group_code, geo_type_code, metro_indicator, education_level_code, education_attendance_code, institution_type_code, employment_status2code, fixed_internet_code, mobile_internet_code, computer_asset_code
Current study status: mostly outside the labour force, kept for description	education_attendance, institution_type
Identifier or survey design column	person_number, psu
Leakage: reveals the employment outcome (log_salary is used in income_data)	reason_not_attending_code, reason_not_attending, lab_salary_raw, salary, salary_status, log_salary
Replaced by age_centred and age_centred_squared	age
Replaced by education_band (31 sparse levels)	education_level
Replaced by female (0/1)	sex
Replaced by its 0/1 version	fixed_internet, mobile_internet, computer_asset
Replaced by settlement_type	geo_type, metro

reason	columns
Replaced by smartphone_access	household_smartphones, smartphones_per_member
Reporting group for Power BI; duplicates age in a model	youth
Target: kept as employed	employment_status
Used to build elderly_in_household and dependency_ratio	n_elderly_65plus, n_working_age

The modelling dataset has 28,742 people from a labour force of 28,846, so 104 people were dropped for missing digital access. 64.9% are employed, a moderate imbalance that Task 4 should keep in mind when choosing the split and the evaluation metrics. The leakage tables show that “Working” as a reason for not studying, and a reported salary, occur almost only among the employed, so these columns are excluded. The only missing values left are in `matric_plus`, for people with “Other/unknown” education; `education_band` and `edu_digital_group` keep these people as their own level. 0 excluded columns are without a documented reason.

T7 Why no scaling is applied

Scaling puts numeric predictors on a common range. It is needed for distance-based or penalised methods such as k-nearest neighbours, k-means or lasso, but not for the methods chosen in Milestone 1, so no scaling is applied. Logistic regression coefficients are simply expressed per unit of each predictor, and tree-based methods split on the order of values, which scaling does not change. `person_weight` is a survey weight and should never be scaled.

```
tibble(
  person_weight_unchanged_since_integration = identical(analytical$person_weight, integrated$person_weight),
  age_centred_range = paste(range(model_data$age_centred), collapse = " to "),
  household_size_range = paste(range(model_data$household_size), collapse = " to ")
)
```

person_weight_unchanged_since_integration	age_centred_range	household_size_range
TRUE	-25 to 24	1 to 25

Numeric predictors keep their natural units (years, people, ratios), so the odds ratios in Milestone 3 stay easy to interpret.

10.5 Encoding for modelling

R's model functions encode categories automatically, but they still need to know which group to compare against. By default, the first factor level is the reference, and the first level of `employment_status` is "Employed", so a logistic regression would predict the chance of being *unemployed*. Ordered factors would also make R use polynomial contrasts, which do not give readable odds ratios. The encoding is therefore set up as follows:

- **E1:** `employed` is a factor with "Unemployed" as the first level, so models predict the probability of being employed.
- **E2:** the yes/no columns are already 0/1 integers (10.2).
- **E3:** each nominal factor gets a meaningful reference level, so every odds ratio reads as "compared with this group":
 - `education_band`: "Some secondary", the largest group below matric.
 - `province`: Gauteng, the largest province and economic hub.
 - `population_group`: Black African, the largest group.
 - `settlement_type`: Metro urban.
 - `edu_digital_group`: "Below matric, no internet", the group with neither resource.
 - `smartphone_access`: "None".
 - `other_member_matric`: "No".
- **E4:** all factors stay unordered, so R uses treatment contrasts (each level compared with the reference).
- **E5:** no manual one-hot encoding is done. `glm()` and `rpart()/randomForest()` accept factors directly and create the dummy columns themselves, as the `model.matrix()` output below shows.
- **E6:** `education_level` is replaced by `education_band` (10.3.1).

```
#fct_relevel() moves the named level to the front, which makes it the reference level
model_data <- model_data |>
  mutate(
    employed = factor(employed, levels = c("Unemployed", "Employed")),

    education_band = fct_relevel(education_band, "Some secondary"),

    province = fct_relevel(province, "Gauteng"),

    population_group = fct_relevel(population_group, "Black African"),
```

```

settlement_type = fct_relevel(settlement_type, "Metro urban"),

edu_digital_group = fct_relevel(edu_digital_group, "Below matric, no internet"),

smartphone_access = fct_relevel(smartphone_access, "None"),

other_member_matric = fct_relevel(other_member_matric, "No")
)

```

For every factor in the modelling dataset, the table shows the reference level, the number of levels, whether it is ordered and the size of its smallest level. The smallest level also checks T5, since no level should be too small to estimate:

```

model_factors <- select(model_data, where(is.factor))

tibble(
  column = names(model_factors),

  reference_level = map_chr(model_factors, ~ levels(.x)[1]),

  number_of_levels = map_int(model_factors, nlevels),

  ordered = map_lgl(model_factors, is.ordered),

  smallest_level_people = map_int(model_factors, ~ min(table(.x)))
)

```

column	reference_level	num- ber_of_levels	or- dered	small- est_level_people
employed	Unemployed	2	FALSE	10101
popula- tion_group	Black African	4	FALSE	551
province	Gauteng	9	FALSE	1163
settlement_type	Metro urban	4	FALSE	1072
education_band	Some secondary	7	FALSE	275
smartphone_ac- cess	None	3	FALSE	2529
edu_digi- tal_group	Below matric,			
no internet	5	FALSE	0	

column	reference_level	num- ber_of_levels	or- dered	small- est_level_people
other_member_matric	No	3	FALSE	4961

```
model_data |> count(employed)
```

employed	n
Unemployed	10101
Employed	18641

```
#E5: R creates one 0/1 column per non-reference level; the reference level (Some secondary) is
colnames(model.matrix(~ education_band, data = model_data))
```

```
[1] "(Intercept)"
[2] "education_bandNo schooling"
[3] "education_bandPrimary (Grade R-7)"
[4] "education_bandMatric"
[5] "education_bandPost-school certificate/diploma"
[6] "education_bandDegree"
[7] "education_bandOther/unknown"
```

```
#Column types in the final modelling dataset
enframe(map_chr(model_data, ~ class(.x)[1]), name = "column", value = "type")
```

column	type
household_id	character
person_weight	numeric
employed	factor
age_centred	numeric
age_centred_squared	numeric
female	integer
population_group	factor
province	factor
settlement_type	factor
education_band	factor
matric_plus	integer

column	type
has_fixed_internet	integer
has_mobile_internet	integer
has_computer	integer
any_home_internet	integer
digital_access_score	integer
smartphone_access	factor
edu_digital_group	factor
household_size	integer
n_children_under15	integer
elderly_in_household	integer
dependency_ratio	numeric
female_x_children	integer
other_member_matric	factor

employed has “Unemployed” as its reference, so a logistic regression models the probability of being employed. Every factor has its chosen reference level first, no factor is ordered, and the `model.matrix()` output shows that R creates the dummy columns itself, leaving out the reference level.

10.6 Saving the analytical datasets

Milestone 3 needs the engineered datasets without having to rerun this pipeline, so three CSV files are saved in `Datasets/Analytical/`:

- `analytical_dataset.csv`: all 45,413 working-age people with every cleaned and engineered column, for Power BI and description.
- `model_data.csv`: the labour-force modelling dataset from 10.4–10.5.
- `income_data.csv`: employed people with a usable salary, for the internet and income question.

As in 8.2, missing values are written as empty cells. A CSV does not store factor levels, so Milestone 3 must read `household_id` as text and re-apply the level order from 10.5 after reading `model_data.csv`.

```
analytical_directory <- "../Datasets/Analytical/"
dir.create(analytical_directory, showWarnings = FALSE)

income_data <- analytical |>
  filter(!is.na(log_salary)) |>
  transmute(
```

```

    household_id,
    person_weight,
    log_salary,
    any_home_internet,
    digital_access_score,
    education_band,
    matric_plus,
    age_centred,
    age_centred_squared,
    female,
    population_group,
    province,
    settlement_type
  )

write_csv(
  analytical,
  file.path(analytical_directory, "analytical_dataset.csv"),
  na = ""
)

write_csv(
  model_data,
  file.path(analytical_directory, "model_data.csv"),
  na = ""
)

write_csv(
  income_data,
  file.path(analytical_directory, "income_data.csv"),
  na = ""
)

```

The saved files are read back with `household_id` as text and compared with the datasets in memory:

```

analytical_saved <- read_csv(
  file.path(analytical_directory, "analytical_dataset.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character(), person_number = col_character())
)

```

```

model_data_saved <- read_csv(
  file.path(analytical_directory, "model_data.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character())
)

income_data_saved <- read_csv(
  file.path(analytical_directory, "income_data.csv"),
  show_col_types = FALSE,
  col_types = cols(household_id = col_character())
)

tibble(
  dataset = c("analytical_dataset", "model_data", "income_data"),

  rows_in_memory = c(
    nrow(analytical),
    nrow(model_data),
    nrow(income_data)
  ),

  rows_in_saved_file = c(
    nrow(analytical_saved),
    nrow(model_data_saved),
    nrow(income_data_saved)
  ),

  same_columns = c(
    identical(names(analytical_saved), names(analytical)),
    identical(names(model_data_saved), names(model_data)),
    identical(names(income_data_saved), names(income_data))),

  ids_identical = c(
    identical(analytical_saved$household_id, analytical$household_id),
    identical(model_data_saved$household_id, model_data$household_id),
    identical(income_data_saved$household_id, income_data$household_id)),

  missing_values_identical = c(
    sum(is.na(analytical_saved)) == sum(is.na(analytical)),
    sum(is.na(model_data_saved)) == sum(is.na(model_data)),
    sum(is.na(income_data_saved)) == sum(is.na(income_data))
  )
)

```

)

dataset	rows_in_memory	rows_in_saved_files	same_columns	ids_identical	missing_values_identical
analytical_dataset	45413	45413	TRUE	TRUE	TRUE
model_data	28742	28742	TRUE	TRUE	TRUE
income_data	18017	18017	TRUE	TRUE	TRUE

Three new files are written to **Datasets/Analytical/**. Read back with the IDs as text, each one has the same rows, columns, IDs and missing values as the dataset in memory. This section does not change the cleaned datasets in **Datasets/Cleaned/** or the original files in **Datasets/**.